

《计算机组成原理实验》

(第五次)

厦门大学信息学院软件工程系

2026年5月

目录

一、验证实验

1. 单总线结构 MIPS 处理器（硬布线控制器）（5条指令）
2. 单总线结构 MIPS 处理器（微程序控制器）（5条指令）
3. 单周期 MIPS 处理器（硬布线控制器）（24条指令）
4. 单周期 RISC-V 处理器（硬布线控制器）（9条指令）

二、设计实验

单总线结构 MIPS 处理器（微程序控制器）（增加1条add指令）（6条指令）

三、挑战性实验

单总线结构 MIPS 处理器（硬布线控制器）（增加1条add指令）（6条指令）

一、验证实验

1. 单总线结构 MIPS 处理器（硬布线控制器）（5条指令）
2. 单总线结构 MIPS 处理器（微程序控制器）（5条指令）
3. 单周期 MIPS 处理器（硬布线控制器）（24条指令）
4. 单周期 RISC-V 处理器（硬布线控制器）（9条指令）

1、单总线结构 MIPS 处理器（硬布线控制器） (5条指令) （验证实验）

- (1) 单总线结构 MIPS 处理器（硬布线控制器）支持的5条指令：

序号	MIPS指令	RTL描述	功能说明
1	lw rt,imm(rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令
2	sw rt,imm(rs)	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存数指令
3	beq rs,rt,imm	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) \ll 2$	条件分支指令：如果rs == rt，则跳转
4	addi rt,rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	立即数加法指令，不考虑溢出
5	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，有符号数比较

- (2) 单总线结构 MIPS 处理器（硬布线控制器）的**数据通路**

参考教材图6.8

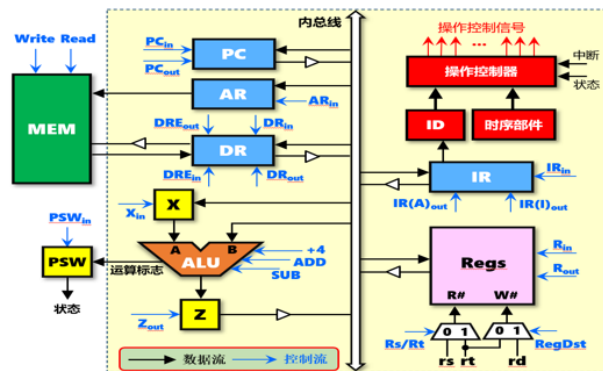
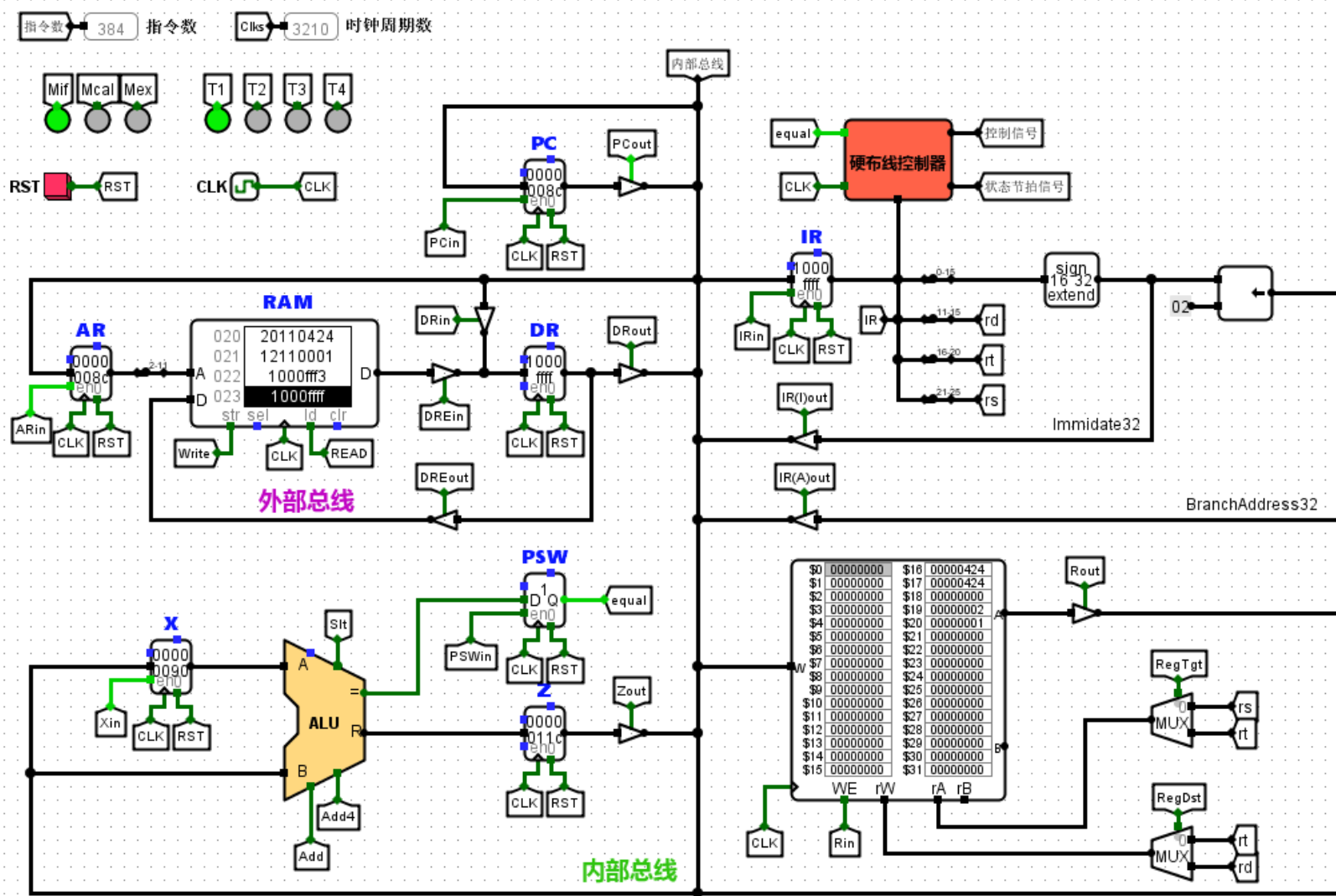


图6.8 单总线结构的计算机框图

三级时序（变长指令周期）

参考教材图6.3

1个指令周期包括：取指周期（Mif）、计算周期（Mcal）、执行周期（Mex）等3个机器周期

1个机器周期包括：2~4个时钟周期（T1、T2、T3、T4）

取指周期：4个时钟周期 计算周期：2个时钟周期 执行周期：3个时钟周期

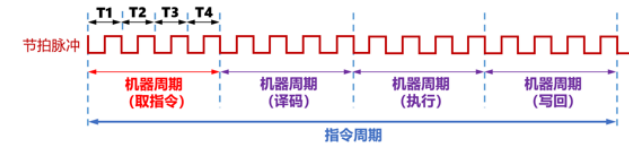
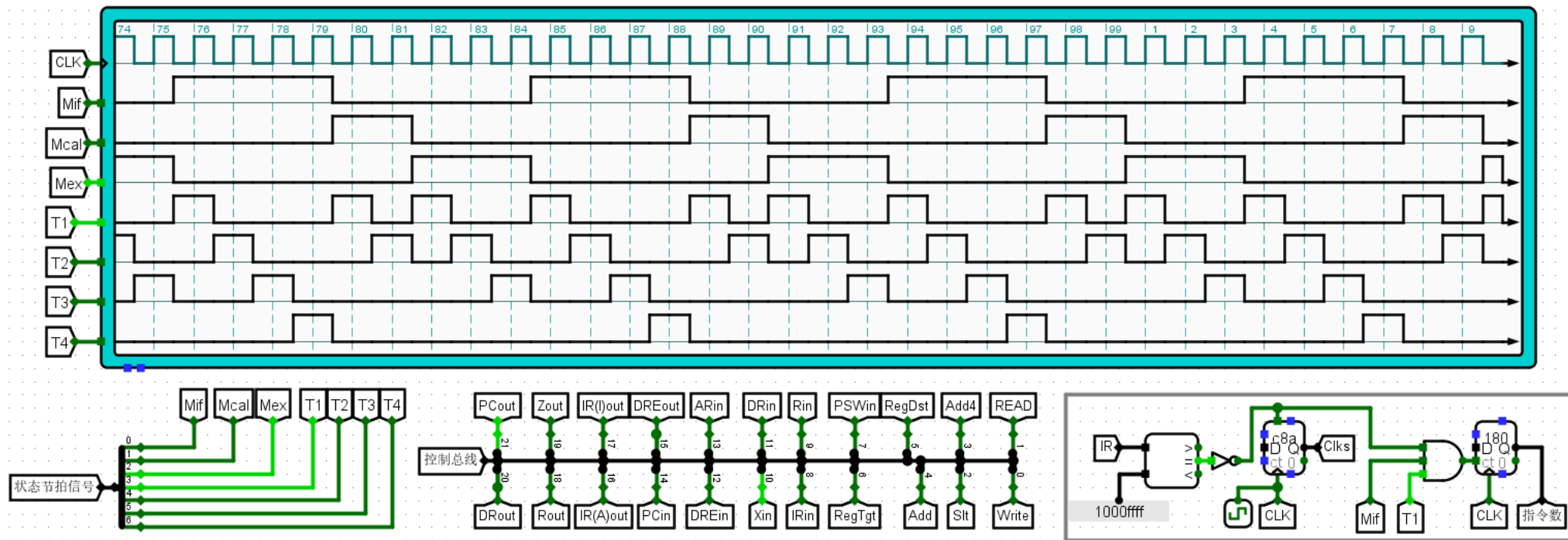


图6.3 指令周期、机器周期与时钟周期的关系



22个控制信号

停机控制

loop2:

beq \$zero,\$zero,loop2

#转loop2

死循环

对应的机器码：1000ffff

硬布线控制器（三级时序、变长指令周期）

硬布线控制器参考教材图6.43

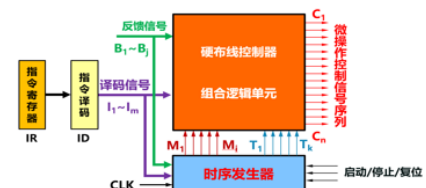
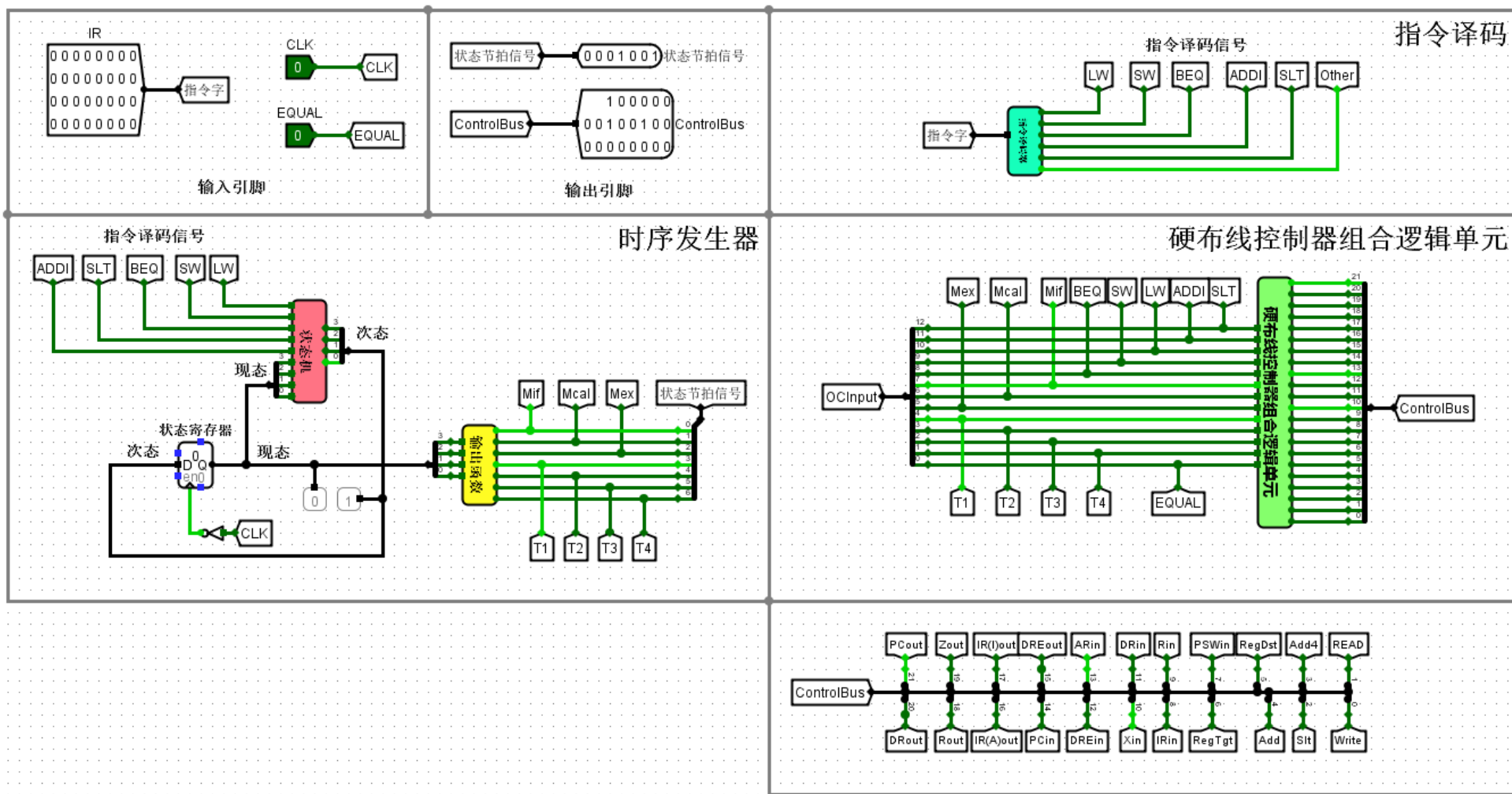


图6.43 传统三级时序硬布线控制器模型



时序发生器参考教材图6.42

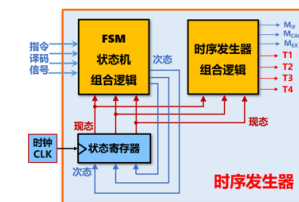
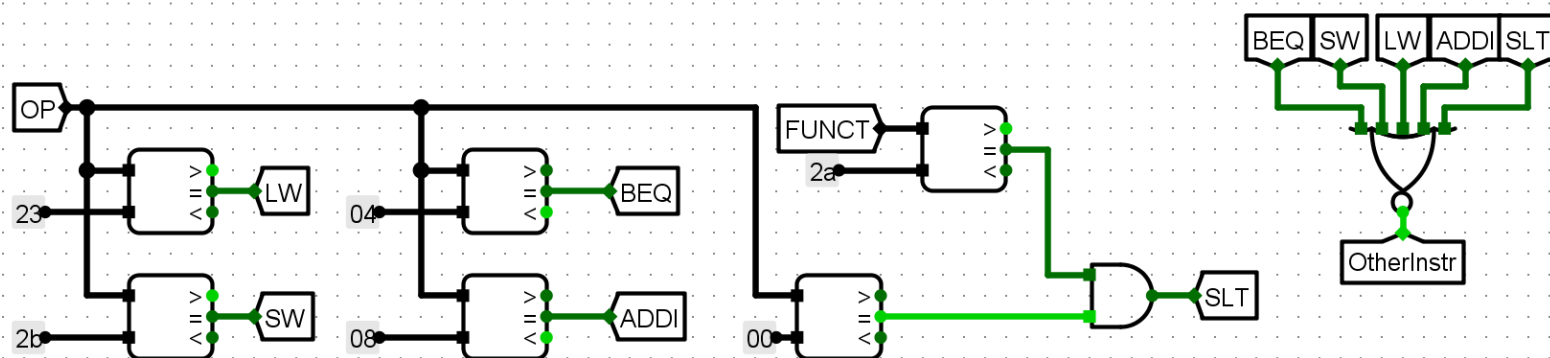
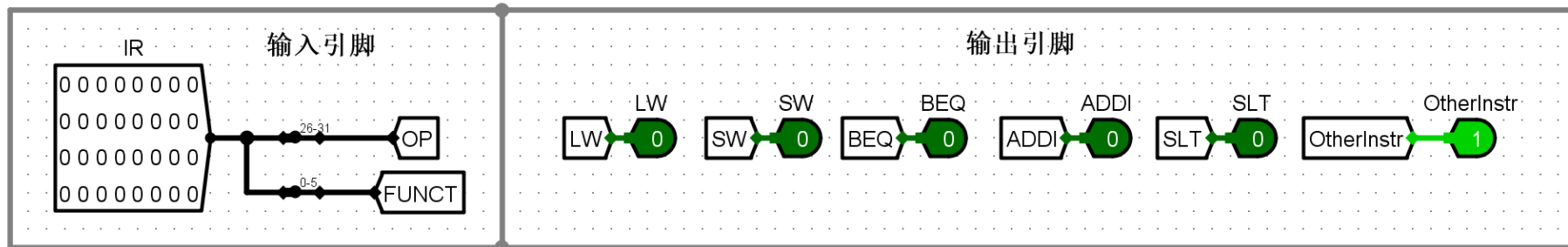


图6.42 时序发生器内部实现逻辑（变长指令周期）

指令译码器（5条指令）

参考教材表5.8、表5.9



IR为LW指令时，指令译码器输出的LW=1，其余为0
 IR为SW指令时，指令译码器输出的SW=1，其余为0
 IR为BEQ指令时，指令译码器输出的BEQ=1，其余为0
 IR为ADDI指令时，指令译码器输出的ADDI=1，其余为0
 IR为SLT指令时，指令译码器输出的SLT=1，其余为0

IR为其他指令时，指令译码器输出的OtherInstr=1，其余为0

表5.8 常用R型指令及其Rtype编码

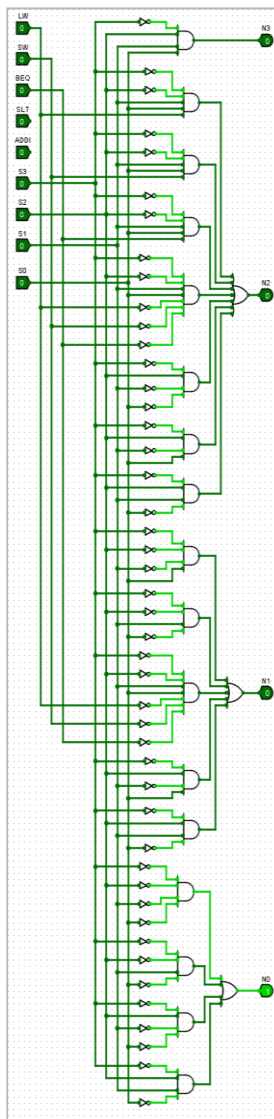
指令名 (Op, 1位)	指令格式	指令功能描述	备注
00	add rd, rs, imm	R[rd] ← R[rs] + imm	寄存器加寄存器，imm为字长未使用
01	addi rd, rs, imm	R[rd] ← R[rs] + imm	寄存器加寄存器，imm为字长未使用
02	sll rd, rs, sh	R[rd] ← R[rs] << sh	寄存器左移寄存器，imm为字长未使用
03	sllr rd, rs, sh	R[rd] ← R[rs] << sh	寄存器左移寄存器
04	srli rd, rs, sh	R[rd] ← R[rs] >> sh	寄存器右移寄存器
05	srrl rd, rs, sh	R[rd] ← R[rs] >> sh	寄存器右移寄存器
06	beq rs, rt, imm	if (R[rs] == R[rt]) PC ← PC + 4 + imm << 2	条件分支指令（相等跳转）
07	bne rs, rt, imm	if (R[rs] != R[rt]) PC ← PC + 4 + imm << 2	条件分支指令（不等跳转）
08	add rd, rs, imm	R[rd] ← R[rs] + imm	立即数加寄存器，imm为字长未使用
09	sll rd, rs, imm	R[rd] ← R[rs] << imm	立即数左移寄存器
10	srl rd, rs, imm	R[rd] ← R[rs] >> imm	立即数右移寄存器
11	and rd, rs, imm	R[rd] ← R[rs] & imm	立即数与寄存器
12	or rd, rs, imm	R[rd] ← R[rs] imm	立即数或寄存器
13	xor rd, rs, imm	R[rd] ← R[rs] ^ imm	立即数异或寄存器
14	slti rd, rs, imm	R[rd] ← (R[rs] < imm) ? 1 : 0	小于立即数指令，有符号比较
15	sltiui rd, rs, imm	R[rd] ← (R[rs] < uiimm) ? 1 : 0	小于立即数指令，无符号比较
16	andi rd, rs, imm	R[rd] ← R[rs] & imm	立即数与寄存器
17	ori rd, rs, imm	R[rd] ← R[rs] imm	立即数或寄存器
18	xori rd, rs, imm	R[rd] ← R[rs] ^ imm	立即数异或寄存器
19	lui rd, imm	R[rd] ← imm << 16	加载立即数指令
20	auipc rd, imm	R[rd] ← PC + imm << 12	加载立即数指令
21	jli rd, imm	R[rd] ← imm << 12	加载立即数指令
22	jal rd, imm	R[rd] ← PC + 4 + imm << 2	加载立即数指令
23	jalr rd, rs, imm	R[rd] ← PC + 4 + R[rs] + imm << 2	加载立即数指令
24	sw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
25	swb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
26	sb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
27	sbw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
28	lh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
29	lhw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
30	lbu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
31	lhu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
32	ld rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
33	ldh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
34	ldw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
35	sd rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
36	sdh rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
37	sdw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
38	sw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
39	swb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
40	sb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
41	sbw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
42	lh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
43	lhw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
44	lbu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
45	lhu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
46	ld rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
47	ldh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
48	ldw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
49	sd rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
50	sdh rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
51	sdw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用

表5.9 常用I型指令及其操作码

Op (6位, 1位)	指令格式	指令功能描述	备注
04	beq rs, rt, imm	if (R[rs] == R[rt]) PC ← PC + 4 + imm << 2	条件分支指令（相等跳转）
05	bne rs, rt, imm	if (R[rs] != R[rt]) PC ← PC + 4 + imm << 2	条件分支指令（不等跳转）
06	add rd, rs, imm	R[rd] ← R[rs] + imm	立即数加寄存器，imm为字长未使用
07	sll rd, rs, imm	R[rd] ← R[rs] << imm	立即数左移寄存器
08	srl rd, rs, imm	R[rd] ← R[rs] >> imm	立即数右移寄存器
09	and rd, rs, imm	R[rd] ← R[rs] & imm	立即数与寄存器
10	ori rd, rs, imm	R[rd] ← R[rs] imm	立即数或寄存器
11	xori rd, rs, imm	R[rd] ← R[rs] ^ imm	立即数异或寄存器
12	andi rd, rs, imm	R[rd] ← R[rs] & imm	立即数与寄存器
13	ori rd, rs, imm	R[rd] ← R[rs] imm	立即数或寄存器
14	xori rd, rs, imm	R[rd] ← R[rs] ^ imm	立即数异或寄存器
15	lui rd, imm	R[rd] ← imm << 16	加载立即数指令
16	auipc rd, imm	R[rd] ← PC + imm << 12	加载立即数指令
17	jli rd, imm	R[rd] ← imm << 12	加载立即数指令
18	jal rd, imm	R[rd] ← PC + 4 + imm << 2	加载立即数指令
19	jalr rd, rs, imm	R[rd] ← PC + 4 + R[rs] + imm << 2	加载立即数指令
20	sw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
21	swb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
22	sb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
23	sbw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
24	lh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
25	lhw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
26	lbu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
27	lhu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
28	ld rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
29	ldh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
30	ldw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
31	sd rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
32	sdh rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
33	sdw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
34	sw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
35	swb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
36	sb rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
37	sbw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
38	lh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
39	lhw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
40	lbu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
41	lhu rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
42	ld rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
43	ldh rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
44	ldw rd, rs, imm	load R[rd], R[rs], imm	寄存器读寄存器，imm为字长未使用
45	sd rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
46	sdh rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用
47	sdw rd, rs, imm	store R[rd], R[rs], imm	寄存器写寄存器，imm为字长未使用

状态机

参考教材图6.14、图6.41



输入：现态（4位），5个指令译码信号（LW、SW、BEQ、SLT、ADDI）

输出：次态（4位）

共9个状态：S0 ~ S8

例如：现态=0000=S0，次态=0001=S1
 现态=0011=S3，SLT=1，次态=0110=S6
 现态=0011=S3，LW=1，次态=0100=S4

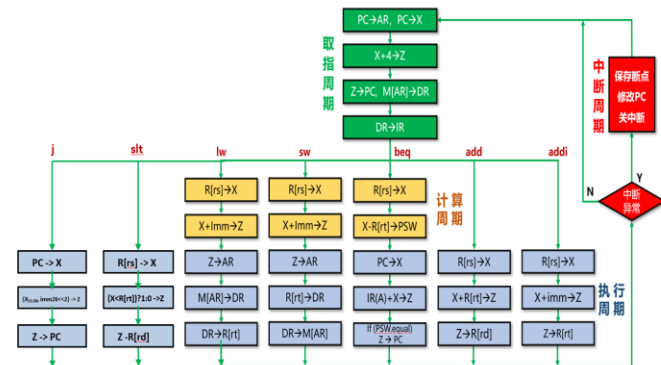
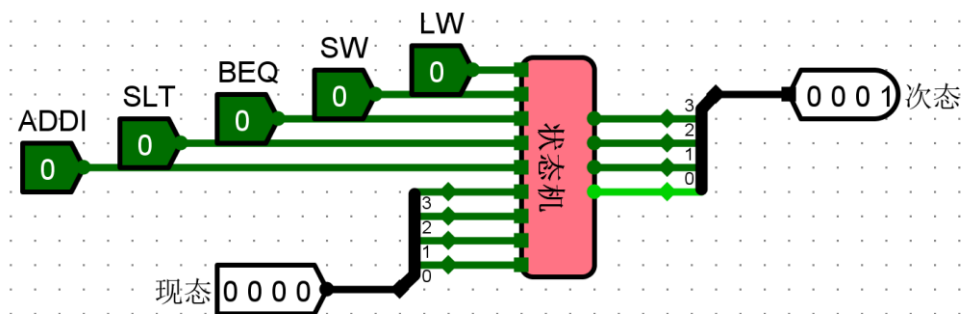


图6.14 单总线结构数据通路指令方框图

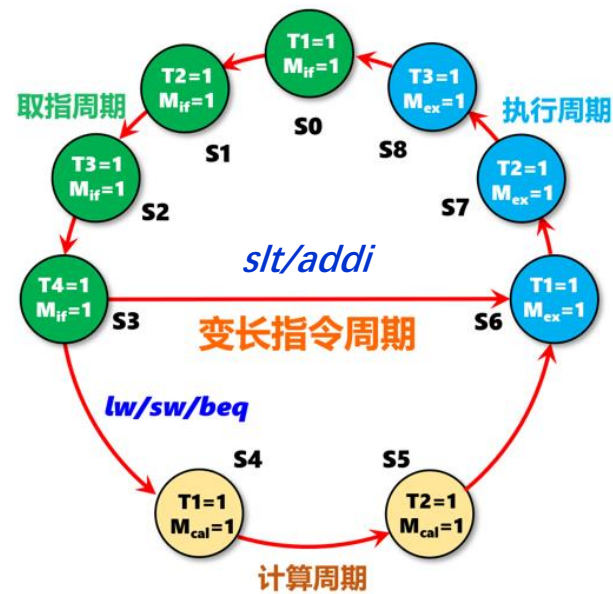
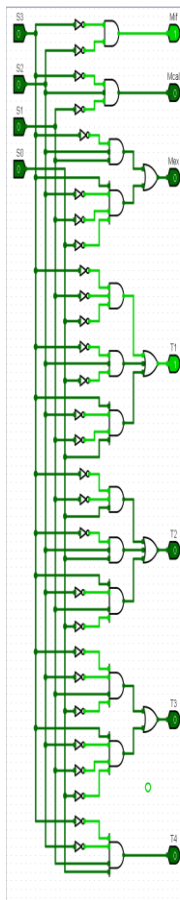


图6.41 变长指令周期三级时序状态机

输出函数

参考教材图6.41



输入：现态（4位）

输出：Mif、Mcal、Mex，T1、T2、T3、T4

例如：现态=0000=S0，输出=Mif、T1
 现态=0001=S1，输出=Mif、T2
 现态=1000=S8，输出=Mex、T3

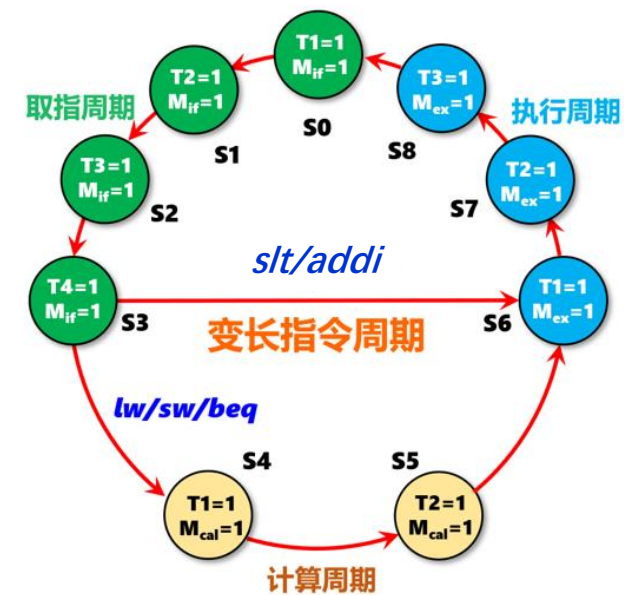
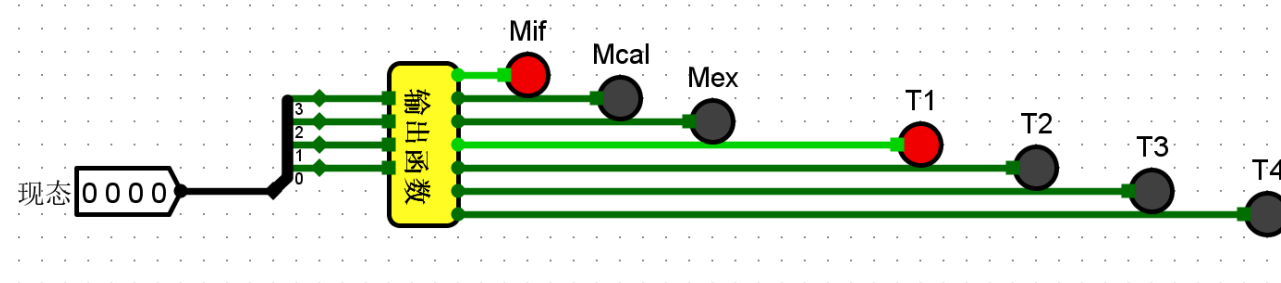


图6.41 变长指令周期三级时序状态机



硬布线控制器组合逻辑单元

参考教材图6.43和6.5.2小节第1部分的内容

输入：Mif、Mcal、Mex, T1、T2、T3、T4, LW、SW、BEQ、SLT、ADDI, EQUAL

输出：22个控制信号

这里的Mif、Mcal、Mex, 相当于图6.43中的 $M_1 \sim M_i$

这里的T1、T2、T3、T4, 相当于图6.43中的 $T_1 \sim T_k$

这里的LW、SW、BEQ、SLT、ADDI, 相当于图6.43中的译码信号 $I_1 \sim I_m$

这里的EQUAL, 相当于图6.43中的反馈信号 $B_1 \sim B_j$

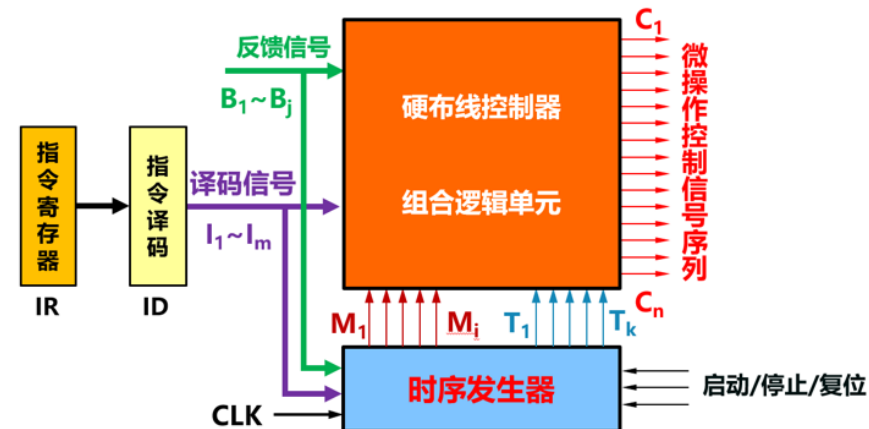
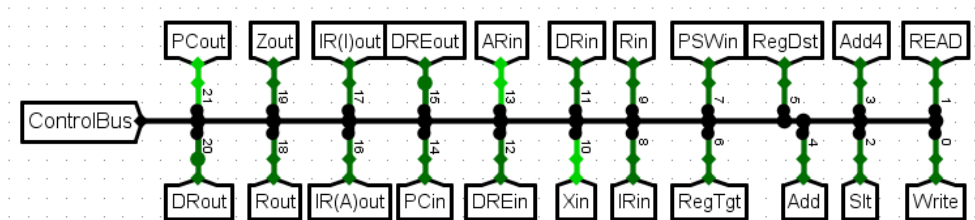
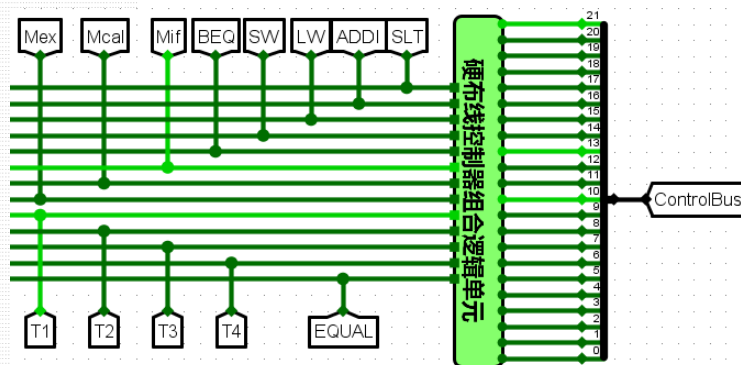
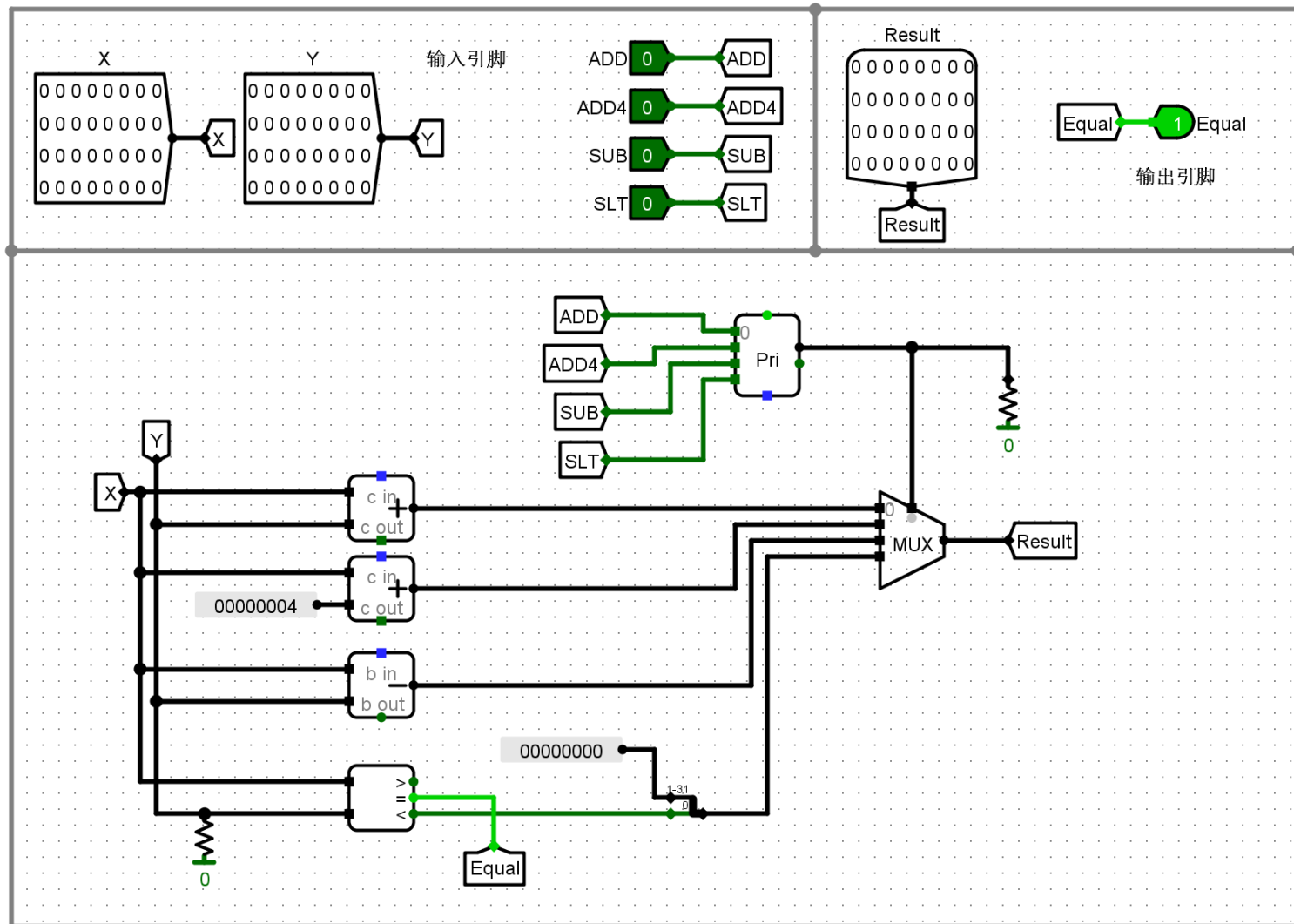


图6.43 传统三级时序硬布线控制器模型



算术逻辑单元ALU

参考第2次实验的内容



ADD=1 Result=X+Y

ADD4=1 Result=X+4

SUB=1 Result=X-Y

SLT=1 Result=(X<Y) ? 1:0

X=Y EQULA=1

- (3) 在单总线结构 MIPS 处理器（硬布线控制器）的数据通路上运行程序：

- 第一步：测试5条指令的程序：test1.asm

test1.asm - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#单总线结构MIPS处理器测试程序

test1.asm

#测试5条指令 (lw、sw、beq、addi、slt)

#程序执行完 RAM存储器100、101、102单元的内容分别为：8、9、1

main:

addi \$s0,\$zero,8 #s0=8
sw \$s0,1024(\$zero) #s0=8 保存到(1024, 即100单元)

addi \$s0,\$zero,9 #s0=9
sw \$s0,1028(\$zero) #s0=9 保存到(1028, 即101单元)

lw \$s1,1024(\$zero) #\$s1=8
lw \$s2,1028(\$zero) #\$s2=9

slt \$s3,\$s1,\$s2 # s1 < s2 s3=1

beq \$s1,\$s2,loop1 #s1不等于s2 执行下一条指令

sw \$s3,1032(\$zero) #s3=1 保存到(1032, 即102单元)

beq \$zero,\$zero,loop2 #转loop2

loop1:

addi \$s4,\$zero,2 #s4=2
sw \$s4,1032(\$zero) #s4=2 保存到(1032, 即102单元)

这条指令不会执行到
这条指令不会执行到

loop2:

beq \$zero,\$zero,loop2 #转loop2 死循环

- 第二步：在单总线结构 MIPS 处理器（硬布线控制器）的数据通路上运行测试程序（**test1.hex**）：
 - 打开Logisim设计文件：**单总线结构MIPS处理器（硬布线控制器）（5条指令）.circ**”
 - 将**test1.hex**装入到RAM中，然后运行程序（先按RST按钮，然后再按Ctrl+K，等到RAM存储器中的内容不动，再按Ctrl+K）
 - 程序运行结束后，观看RAM存储器**100 ~ 102**存储单元中的内容是否正确

```
100 00000008 00000009 00000001
```

- 第三步：运行5个数的排序程序**sort3_mips_bus.hex**和**sort4_mips_bus.hex**
 - 程序运行结束后，观看RAM存储器**100 ~ 104**存储单元中的内容是否正确

```
100 00000005 00000004 00000003 00000002 00000001
```

```
100 00000001 00000002 00000003 00000004 00000005
```

#MIPS程序 sort3_mips_bus.asm 排序程序 (降序排列, 从大到小) 仅使用5条指令 (lw, sw, beq, addi, slt)

#先将5个数 (3、1、5、2、4) 存放到地址为1024开始的RAM存储器中, 然后对这5个数进行排序, 排序好的5个数 (5、4、3、2、1) 仍然放在地址为1024开始的RAM存储器中

.text

main:

```
addi $s0,$zero,3      #第1个数=3 (可以修改) 保存到(1024+0)
sw $s0,1024($zero)
addi $s0,$zero,1      #第2个数=1 (可以修改) 保存到(1024+4)
sw $s0,1028($zero)
addi $s0,$zero,5      #第3个数=5 (可以修改) 保存到(1024+8)
sw $s0,1032($zero)
addi $s0,$zero,2      #第4个数=2 (可以修改) 保存到(1024+12)
sw $s0,1036($zero)
addi $s0,$zero,4      #第5个数=4 (可以修改) 保存到(1024+16)
sw $s0,1040($zero)
```

```
addi $s0,$zero,1024    # $s0=1024          排序区间开始地址
addi $s1,$zero,1040    # $s1=1040=1024+5*4-4  排序区间结束地址    5个数
```

sort_loop:

```
lw $s3,0($s0)          # $s3=($s0)
lw $s4,0($s1)          # $s4=($s1)
slt $t0,$s3,$s4        # 如果 $s3 < $s4, 则置 $t0=1; 否则, 置 $t0=0    降序排序    从大到小
beq $t0,$zero,sort_next # 如果 $t0=0, 则转sort_next
sw $s3,0($s1)          # 交换($s0)和($s1)
sw $s4,0($s0)          # 交换($s0)和($s1)
```

sort_next:

```
addi $s1,$s1,-4        # $s1-4 -> $s1
beq $s0,$s1,loop1      # 如果 $s0=$s1, 则转loop1
beq $zero,$zero,sort_loop # 转sort_loop
```

loop1:

```
addi $s0,$s0,4          # $s0+4 -> $s0
addi $s1,$zero,1040     # $s1=1040=1024+5*4-4  排序区间结束地址    5个数
beq $s0,$s1,loop2      # 如果 $s0=$s1, 则转loop2
beq $zero,$zero,sort_loop # 转sort_loop
```

loop2:

```
beq $zero,$zero,loop2   # 转loop2    死循环
```


#MIPS程序 sort4_mips_bus.asm 排序程序 (升序排列, 从小到大) 仅使用5条指令 (lw、sw、beq、addi、slt)

#先将5个数 (3、1、5、2、4) 存放到地址为1024开始的RAM存储器中, 然后对这5个数进行排序, 排序好的5个数 (1、2、3、4、5) 仍然放在地址为1024开始的RAM存储器中

.text

main:

```
addi $s0,$zero,3      #第1个数=3 (可以修改) 保存到(1024+0)
sw $s0,1024($zero)
addi $s0,$zero,1      #第2个数=1 (可以修改) 保存到(1024+4)
sw $s0,1028($zero)
addi $s0,$zero,5      #第3个数=5 (可以修改) 保存到(1024+8)
sw $s0,1032($zero)
addi $s0,$zero,2      #第4个数=2 (可以修改) 保存到(1024+12)
sw $s0,1036($zero)
addi $s0,$zero,4      #第5个数=4 (可以修改) 保存到(1024+16)
sw $s0,1040($zero)
```

```
addi $s0,$zero,1024   # $s0=1024      排序区间开始地址
addi $s1,$zero,1040   # $s1=1040=1024+5*4-4  排序区间结束地址    5个数
```

sort_loop:

```
lw $s3,0($s0)         # $s3=($s0)
lw $s4,0($s1)         # $s4=($s1)
slt $t0,$s4,$s3       # 如果 $s4 < $s3, 则置 $t0=1; 否则, 置 $t0=0    升序排序    从小到大
beq $t0,$zero,sort_next # 如果 $t0=0, 则转sort_next
sw $s3,0($s1)         # 交换($s0)和($s1)
sw $s4,0($s0)         # 交换($s0)和($s1)
```

sort_next:

```
addi $s1,$s1,-4       # $s1-4 -> $s1
beq $s0,$s1,loop1     # 如果 $s0=$s1, 则转loop1
beq $zero,$zero,sort_loop # 转sort_loop
```

loop1:

```
addi $s0,$s0,4        # $s0+4 -> $s0
addi $s1,$zero,1040   # $s1=1040=1024+5*4-4  排序区间结束地址    5个数
beq $s0,$s1,loop2     # 如果 $s0=$s1, 则转loop2
beq $zero,$zero,sort_loop # 转sort_loop
```

loop2:

```
beq $zero,$zero,loop2 # 转loop2      死循环
```

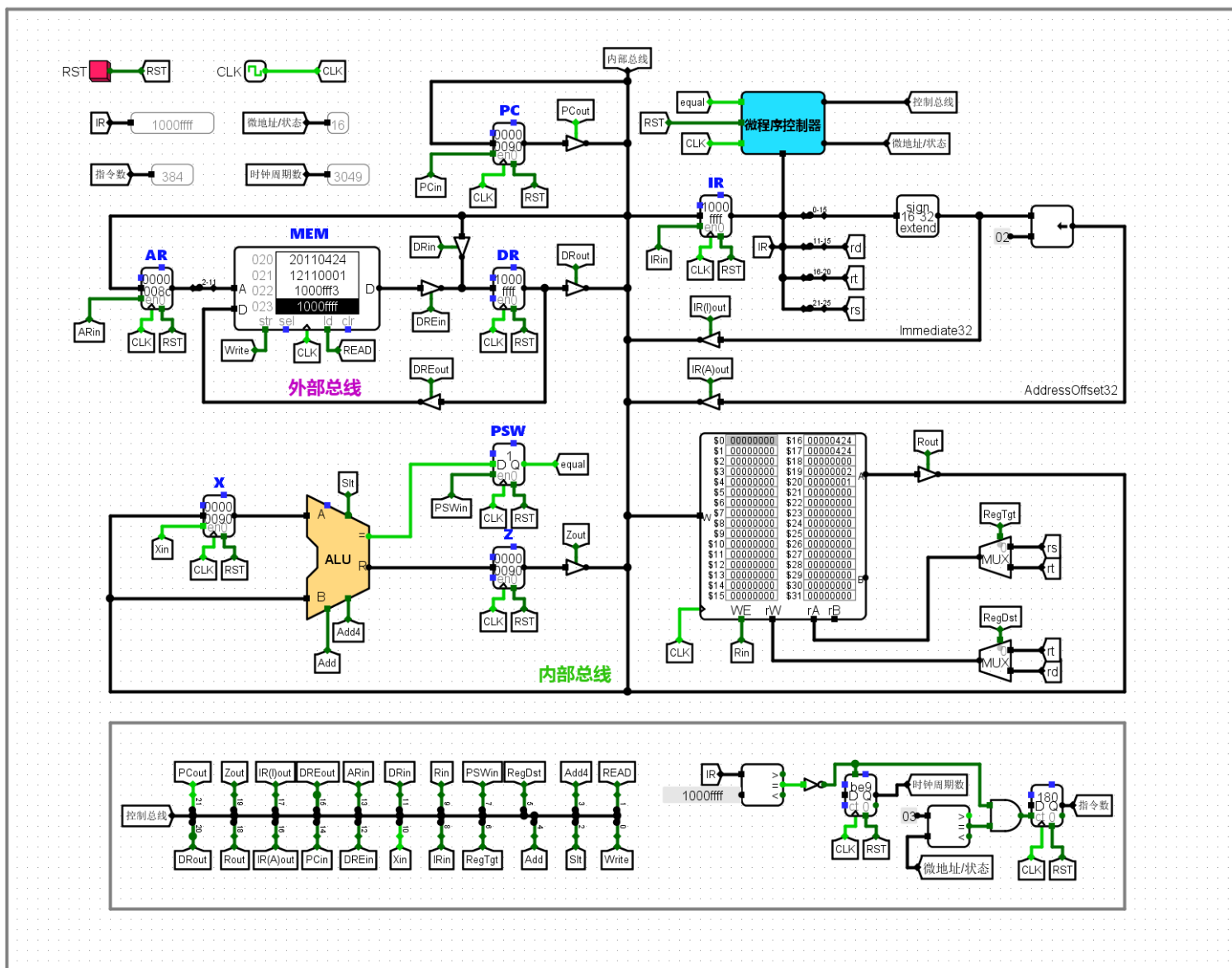

- (4) 请问：在该单总线结构 MIPS 处理器（硬布线控制器）的数据通路上，能够运行**求累加和程序、计算费波那契数列程序**吗？为什么？
- (5) 请分析单总线结构 MIPS 处理器（硬布线控制器）的**电路原理**：
 - 包括：数据通路、硬布线控制器、指令译码器、状态机、输出函数、硬布线控制器组合逻辑单元、算术逻辑单元ALU等电路。

2、单总线结构 MIPS 处理器（微程序控制器） (5条指令) （验证实验）

- （1）单总线结构 MIPS 处理器（微程序控制器）支持的**5条指令**：

序号	MIPS指令	RTL描述	功能说明
1	lw rt,imm(rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm})]$	取数指令
2	sw rt,imm(rs)	$M[R[rs] + \text{SignExt}(\text{imm})] \leftarrow R[rt]$	存数指令
3	beq rs,rt,imm	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(\text{imm}) \ll 2$	条件分支指令：如果rs == rt，则跳转
4	addi rt,rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(\text{imm})$	立即数加法指令，不考虑溢出
5	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，有符号数比较

- (2) 单总线结构 MIPS 处理器（微程序控制器）的**数据通路**（与前面的**硬布线控制器**相同）



微程序控制器（下址字段法）

参考教材图6.51

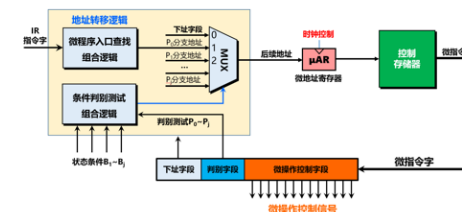
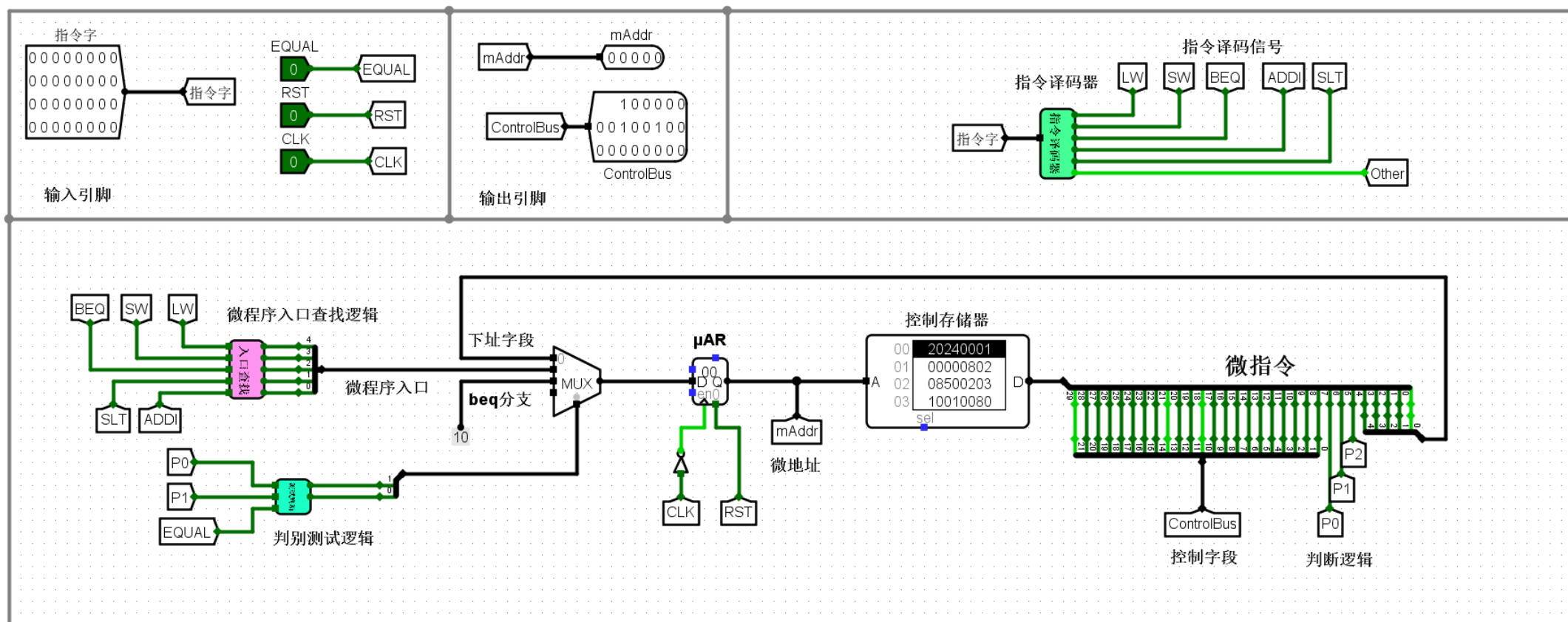
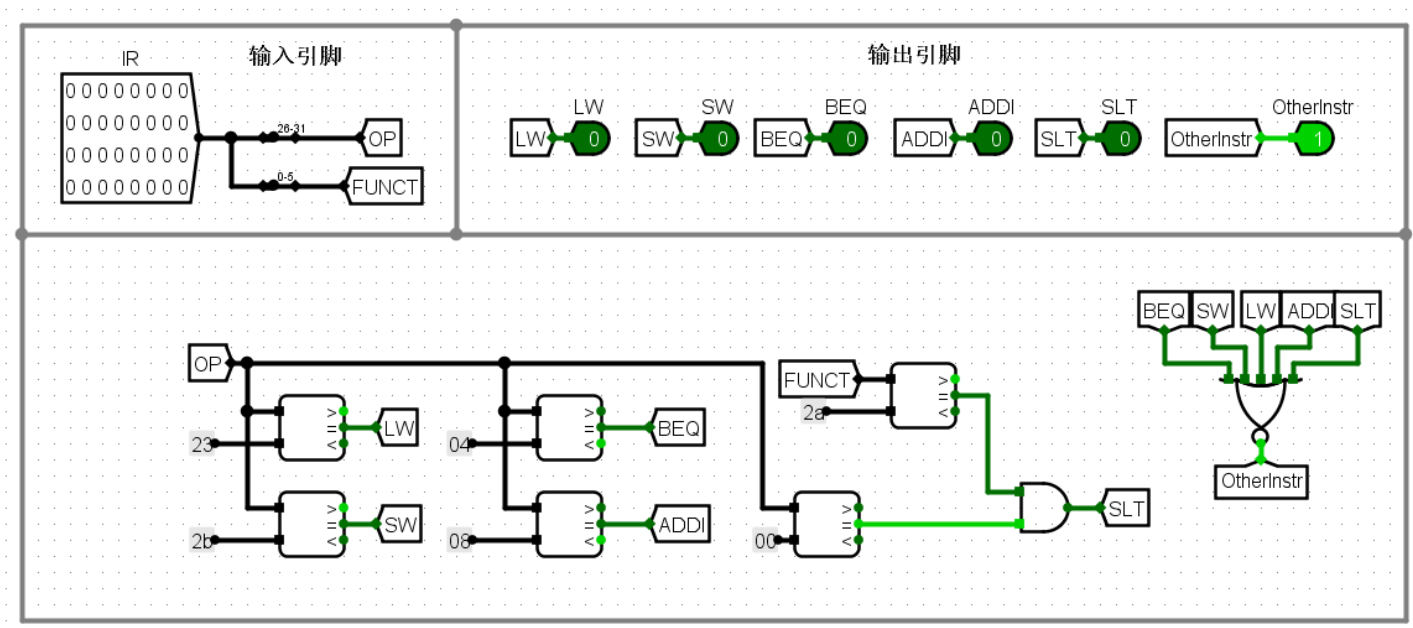


图6.51 地址转移逻辑（下址字段法）



指令译码器（5条指令）（与硬布线控制器相同）



IR为LW指令时，指令译码器输出的LW=1，其余为0

IR为SW指令时，指令译码器输出的SW=1，其余为0

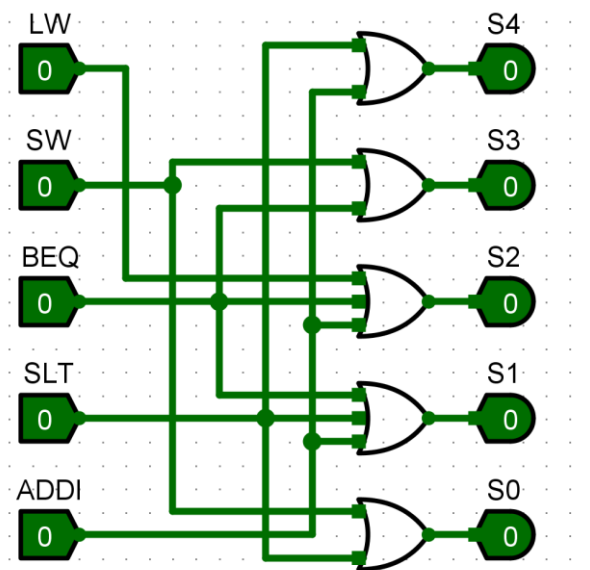
IR为BEQ指令时，指令译码器输出的BEQ=1，其余为0

IR为ADDI指令时，指令译码器输出的ADDI=1，其余为0

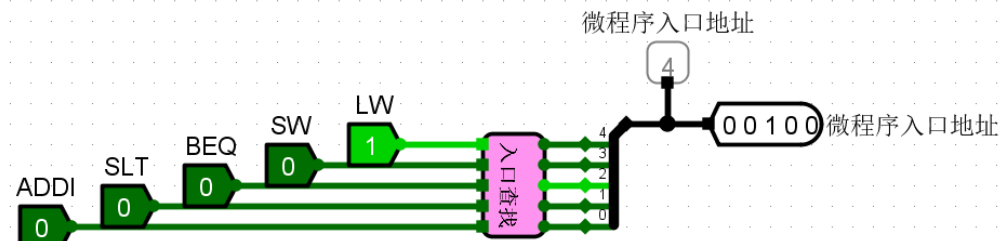
IR为SLT指令时，指令译码器输出的SLT=1，其余为0

IR为其他指令时，指令译码器输出的OtherInstr=1，其余为0

微程序入口查找逻辑



微程序入口查找逻辑



参考教材图6.45

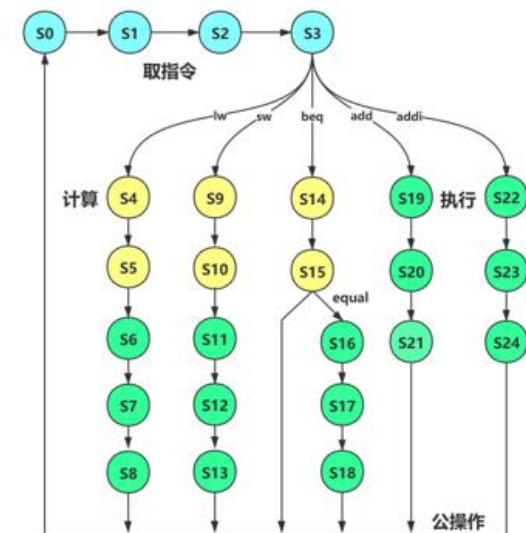


图6.45 指令执行状态转换图（单总线结构处理器）

LW=1	微程序入口地址=4
SW=1	微程序入口地址=9
BEQ=1	微程序入口地址=14
SLT=1	微程序入口地址=19
ADDI=1	微程序入口地址=22

判别测试逻辑

P1与Equal信号的组合状态一共有4种：

- 00：顺序
- 01：顺序
- 10：顺序
- 11：分支跳转

P0即 P_{IR} ，P1即 P_{equal}

P0=1 P1=任意 EQUAL=任意

M1M0=01

微程序入口

P0=0 P1=1 EQUAL=1

M1M0=10

beq分支=10H=16

P0=0 P1=0 EQUAL=0

M1M0=00

下址字段

P0=0 P1=0 EQUAL=1

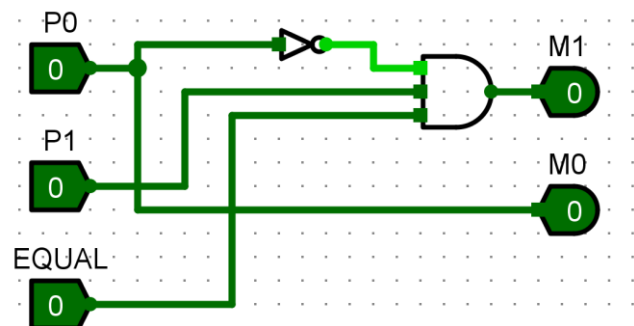
M1M0=00

下址字段

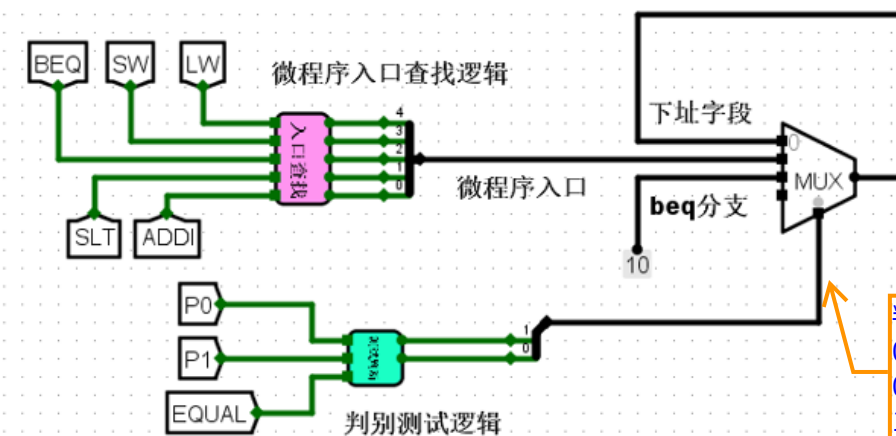
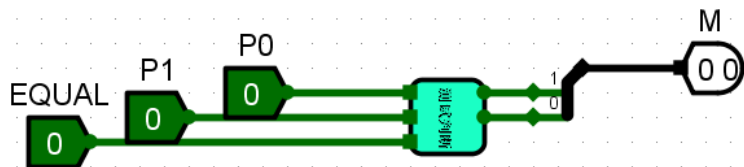
P0=0 P1=1 EQUAL=0

M1M0=00

下址字段



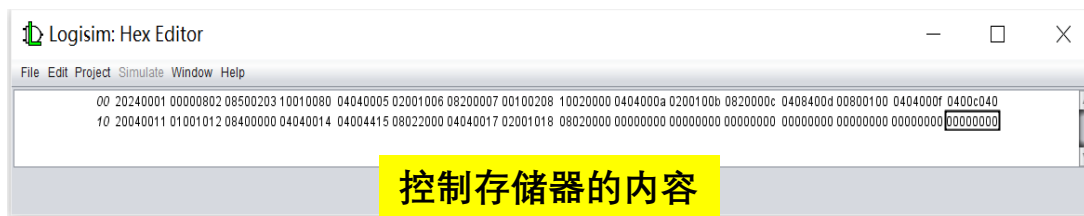
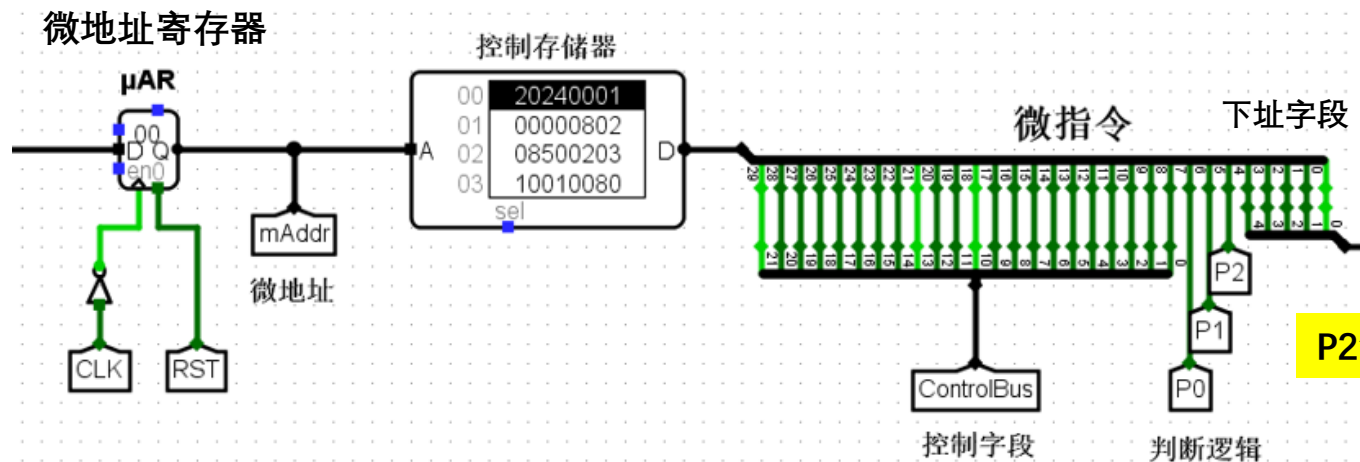
判别测试逻辑



判别测试逻辑输出：
00：MUX输出下址字段
01：MUX输出微程序入口
10：MUX输出beq分支地址

控制存储器（存放25条微指令）

微指令格式（30位）=控制字段（22位）+判断逻辑（3位）+下址字段（5位）



微程序（单总线）

文件(F) 编辑(E) 格式(O)

v2.0 raw

20240001

802

8500203

10010080

4040005

2001006

8200007

100208

10020000

404000A

200100B

820000C

408400D

800100

404000F

400C040

20040011

1001012

8400000

4040014

4004415

8022000

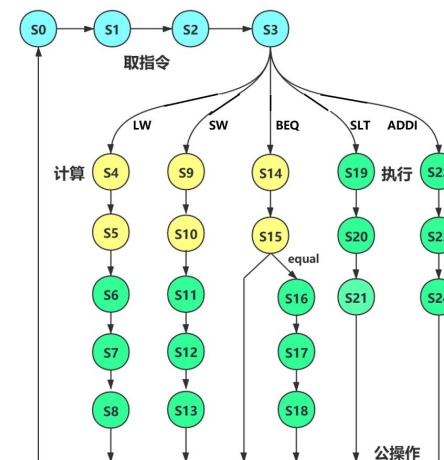
4040017

2001018

8020000

微程序

25条微指令



- (3) 请在单总线结构 MIPS 处理器（微程序控制器）的数据通路上**运行程序**：
 - test1.hex、sort3_mips_bus.hex、sort4_mips_bus.hex
 - 具体步骤与单总线结构 MIPS 处理器（硬布线控制器）相同。
 - 将.hex文件装入到RAM中，按RST按钮，再按Ctrl+K，等到RAM存储器中的内容不动，再按Ctrl+K。
 - 观看RAM存储器中的结果是否正确？

100 00000008 00000009 00000001

100 00000005 00000004 00000003 00000002 00000001

100 00000001 00000002 00000003 00000004 00000005

- (4) 请问：在该单总线结构 MIPS 处理器（微程序控制器）的数据通路上，能够运行**求累加和程序、计算费波那契数列程序**吗？为什么？
- (5) 请分析单总线结构 MIPS 处理器（微程序控制器）的**电路原理和微程序**：
 - 包括：数据通路、微程序控制器（下址字段法）、微程序入口查找逻辑、判别测试逻辑、控制存储器中的微程序（25条微指令）等。

3、单周期 MIPS 处理器（硬布线控制器） (24条指令) （验证实验）

第四次实验的内容

- （1）单周期 MIPS 处理器（硬布线控制器）支持的24条指令：

核心指令（8条）

序号	MIPS指令	RTL描述	功能说明
1	add \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	寄存器加法指令，不考虑溢出
2	slt rd,rs,rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，有符号数比较
3	addi rt,rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(\text{imm})$	立即数加法指令，不考虑溢出
4	lw rt,imm(rs)	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm})]$	取数指令
5	sw rt,imm(rs)	$M[R[rs] + \text{SignExt}(\text{imm})] \leftarrow R[rt]$	存数指令
6	beq rs,rt,imm	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(\text{imm}) \ll 2$	条件分支指令：如果rs == rt，则跳转
7	bne rs,rt,imm	$\text{if}(R[rs] != R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(\text{imm}) \ll 2$	条件分支指令：如果rs != rt，则跳转
8	syscall	系统调用指令，用于停机	系统调用指令，用于停机

基础指令（16条）

序号	MIPS指令	RTL描述	功能说明
1	addu \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] + R[rt]$	无符号寄存器加法指令
2	addiu \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	无符号立即数加法指令
3	and \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] \& R[rt]$	逻辑与指令
4	andi \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \& \{16\text{个}0, imm\}$	立即数逻辑与指令
5	sll \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \ll \text{shamt}$	逻辑左移
6	srl \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	逻辑右移
7	sra \$rd,\$rt,imm	$R[rd] \leftarrow R[rt] \gg \text{shamt}$	算术右移
8	sub \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] - R[rt]$	寄存器减法指令
9	or \$rd,\$rs,\$rt	$R[rd] \leftarrow R[rs] R[rt]$	逻辑或指令
10	ori \$rt,\$rs,imm	$R[rt] \leftarrow R[rs] \{16\text{个}0, imm\}$	立即数逻辑或指令
11	nor \$rd,\$rs,\$rt	$R[rd] \leftarrow \neg(R[rs] R[rt])$	逻辑或非指令
12	sltu \$rd,\$rs,\$rt	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	小于置1指令，无符号数比较
13	slti \$rt,\$rs,imm	$R[rd] \leftarrow (R[rs] < \text{SignExt}(imm)) ? 1:0$	小于置1指令，立即数比较
14	j address	$PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	无条件分支指令
15	jal address	$R[31] \leftarrow PC+4 \quad PC \leftarrow \{(PC+4)_{31:28}, address<<2\}$	子程序调用指令
16	jr \$rs	$PC \leftarrow R[\$rs]$	寄存器跳转指令

- (2) 单周期 MIPS 处理器（硬布线控制器）的数据通路

参考教材图6.25

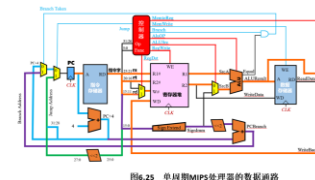


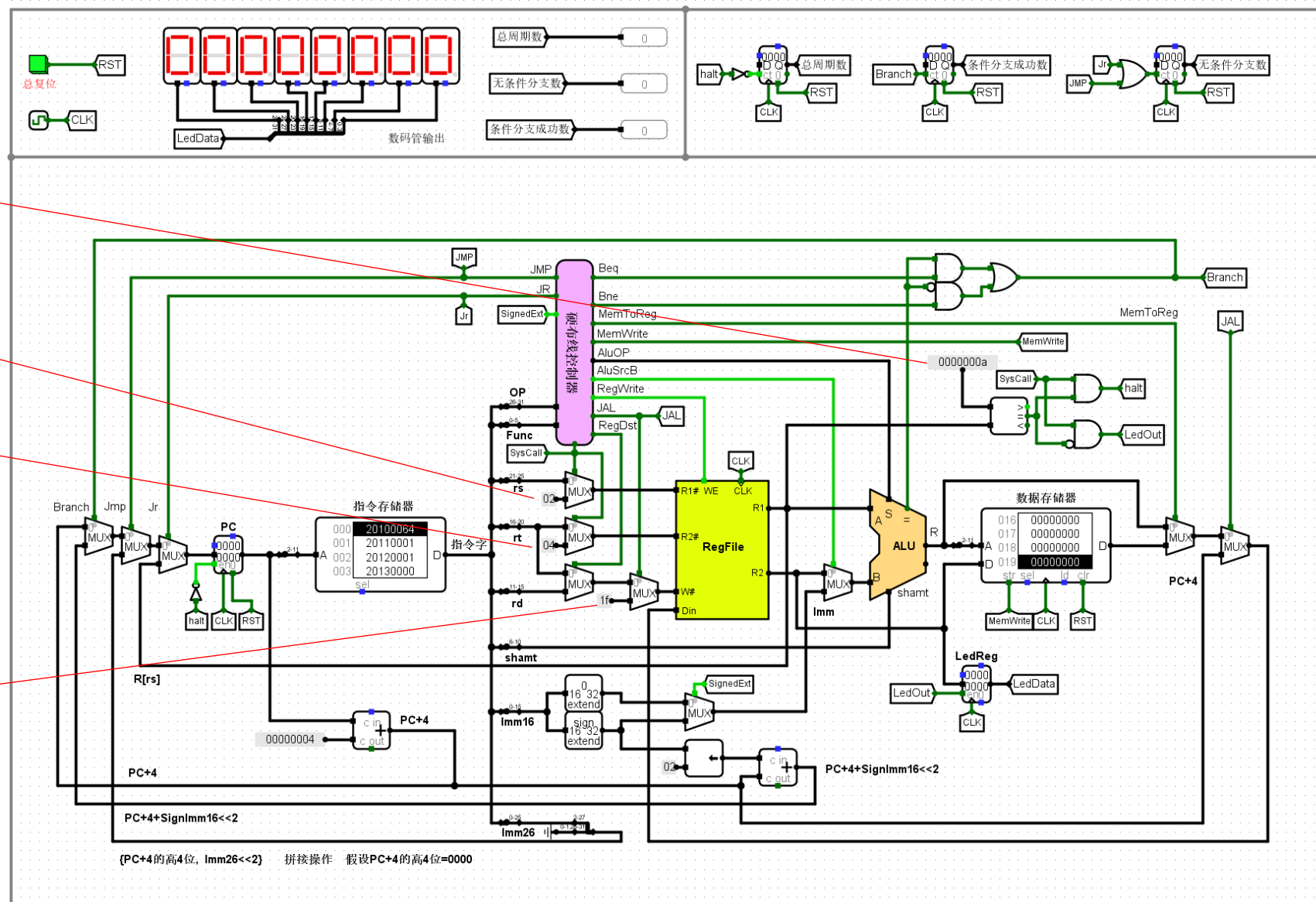
图6.25 单周期MIPS处理器的数据通路

a(10): syscall指令的10号功能（停机）

02: 即2号寄存器（v0），用于syscall指令

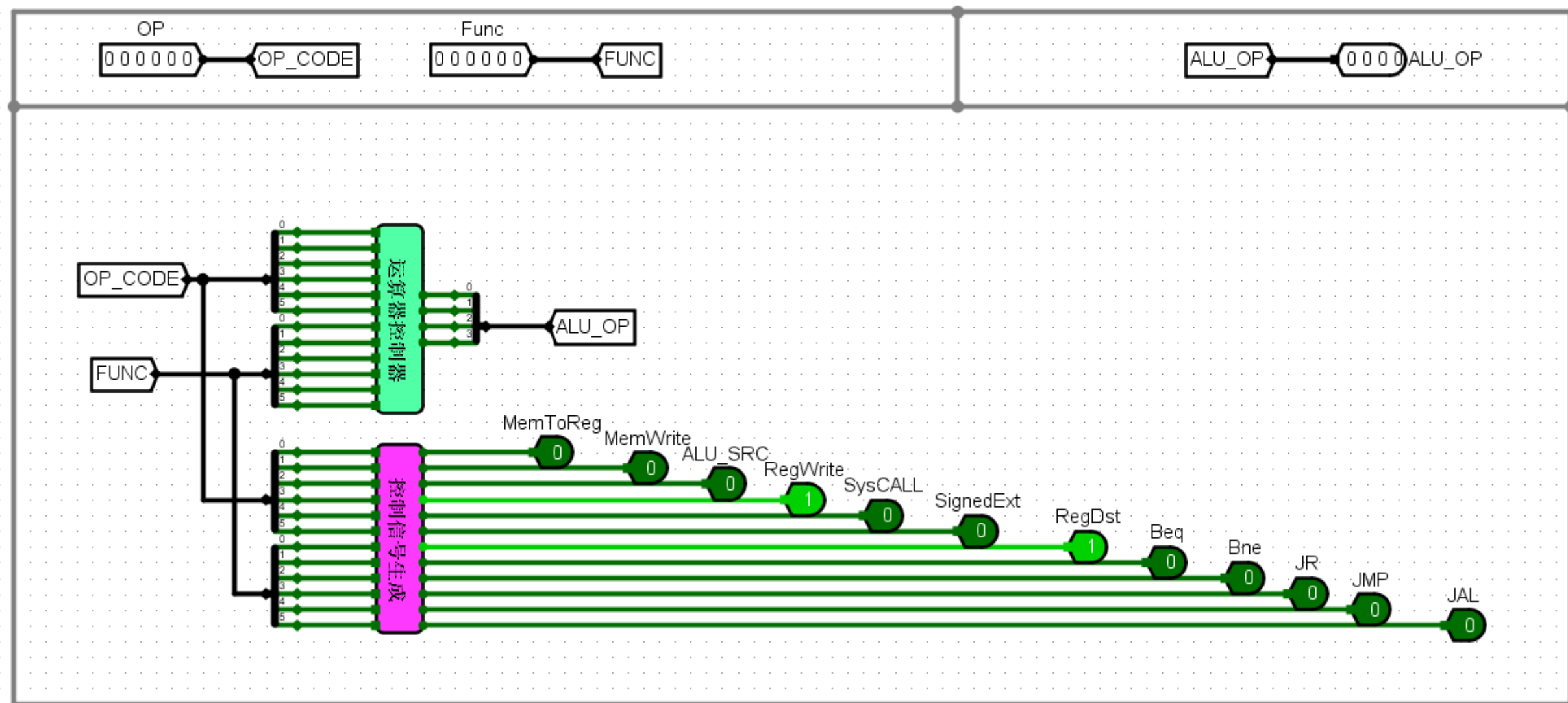
04: 即4号寄存器（a0），用于syscall指令

1f(31): 即31号寄存器（ra），用于jal指令（子程序调用指令）



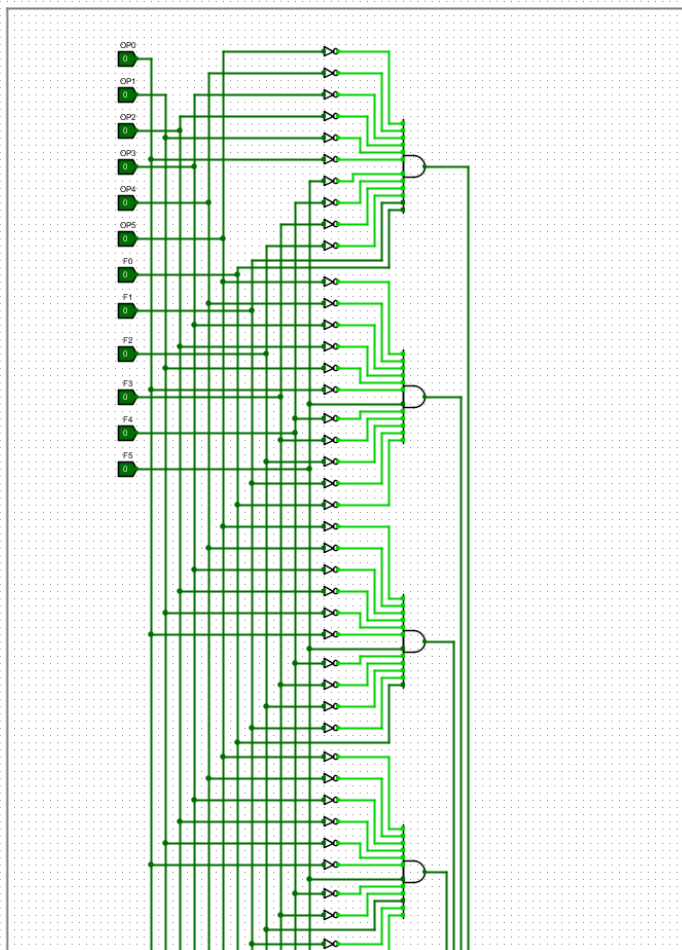
硬布线控制器

参考教材图6.25中的控制器



运算器控制器

运算器控制器



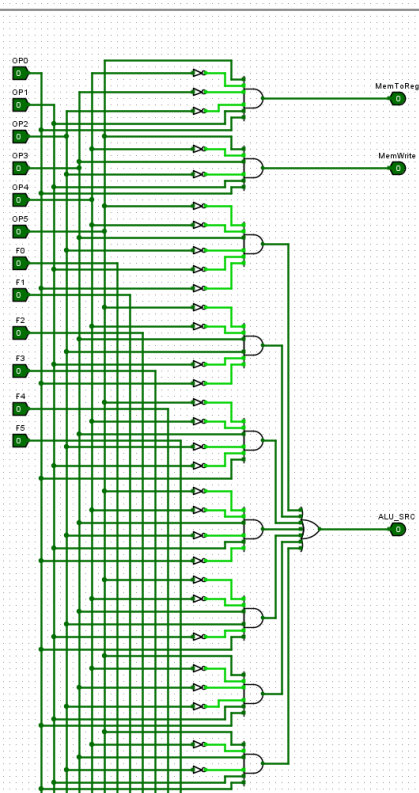
输入：OP（6位）、funct（6位）

输出：AluOP（4位），13种运算

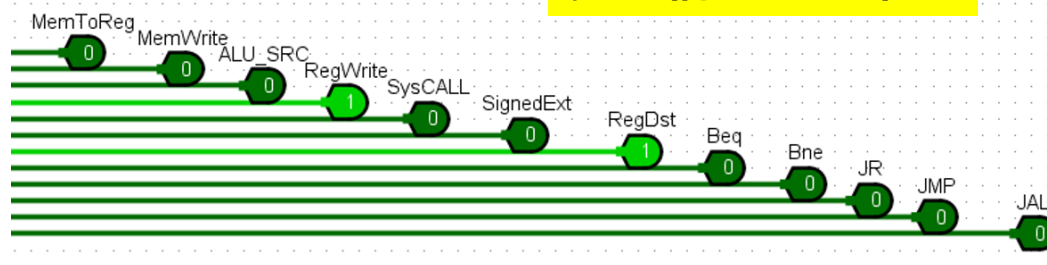
根据指令的OP字段（6位）、funct字段（6位），知道是哪条运算类指令，然后给出运算器的运算功能AluOP（4位）（13种运算）

控制信号生成

控制信号生成



控制信号（12个）



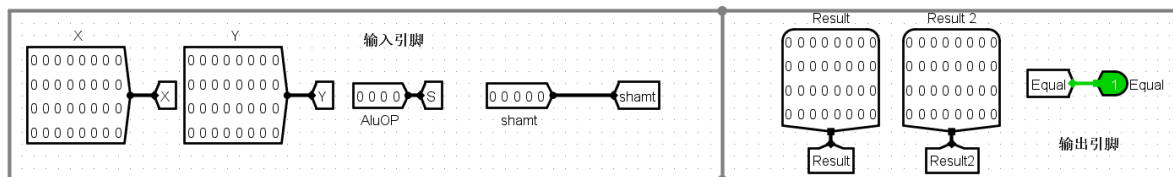
输入：OP（6位）、func（6位）

输出：控制信号（12个）

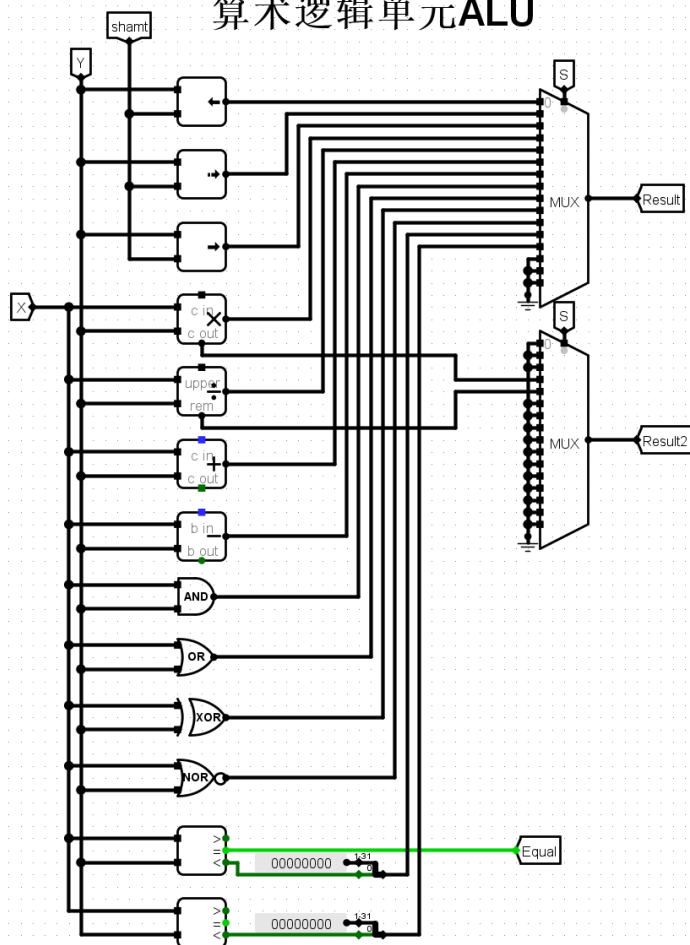
根据指令的OP字段（6位）、func字段（6位），知道是哪条指令，然后给出相应的控制信号（12个）

ALU

参考第2次实验的内容



算术逻辑单元ALU



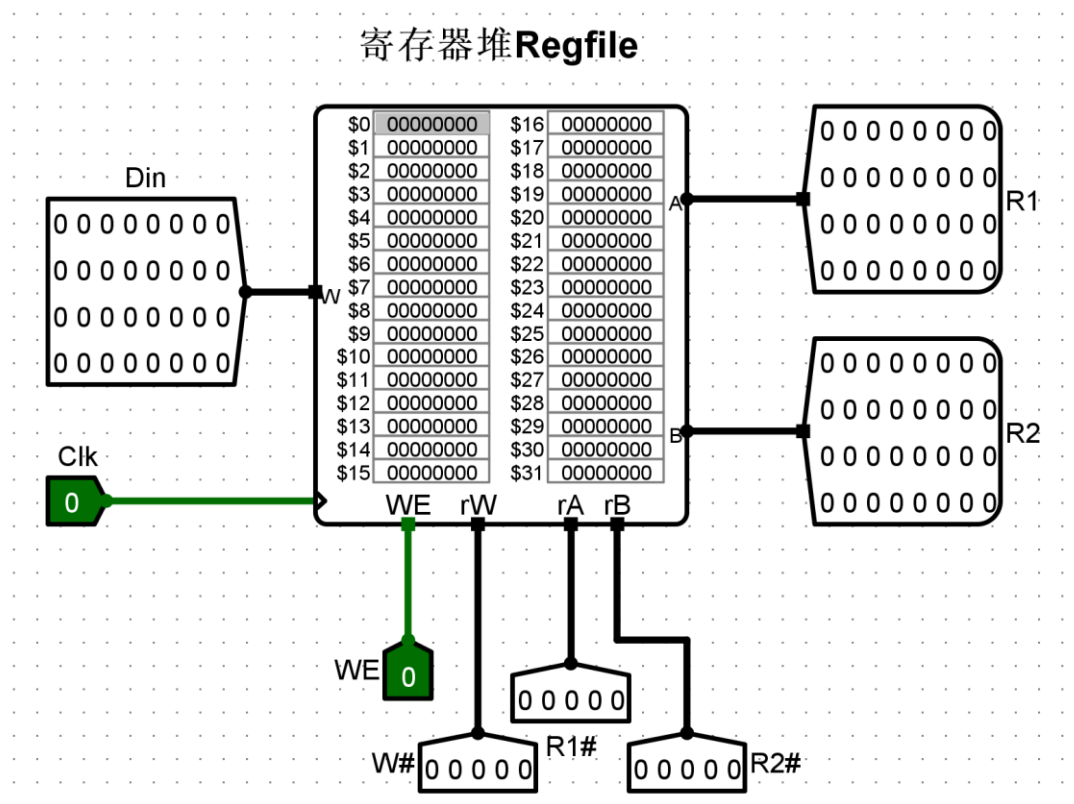
13种运算

AluOP=0000	逻辑左移
AluOP=0001	算术右移
AluOP=0010	逻辑右移
AluOP=0011	乘法
AluOP=0100	除法
AluOP=0101	加法
AluOP=0110	减法
AluOP=0111	逻辑与
AluOP=1000	逻辑或
AluOP=1001	逻辑异或
AluOP=1010	逻辑或非
AluOP=1011	有符号数比较
AluOP=1100	无符号数比较

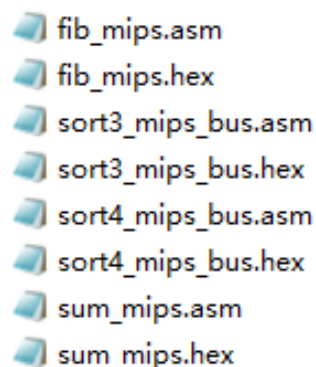
X=Y EQUAL=1

寄存器堆 (Regfile)

参考第3次实验的内容



- (3) **测试程序**：包括求累加和程序、计算费波那契数列程序、排序程序（降序、升序）
 - 先在指令存储器中装入.hex文件，按Ctrl+K，再按“总复位”按钮，指令存储器中的内容停止后，按Ctrl+K，观看数据存储器中的内容是否正确？



fib_mips.asm
fib_mips.hex
sort3_mips_bus.asm
sort3_mips_bus.hex
sort4_mips_bus.asm
sort4_mips_bus.hex
sum_mips.asm
sum_mips.hex

- (4) 请问：该单周期 MIPS 处理器的控制器能不能采用**微程序控制器**的方法设计？为什么？
- (5) 请分析单周期 MIPS 处理器（硬布线控制器）的**电路原理**：
 - 包括：数据通路、硬布线控制器、运算器控制器、控制信号生成等电路。
- (6) 鼓励同学们编写**测试程序**，对**24条指令**的功能进行测试。

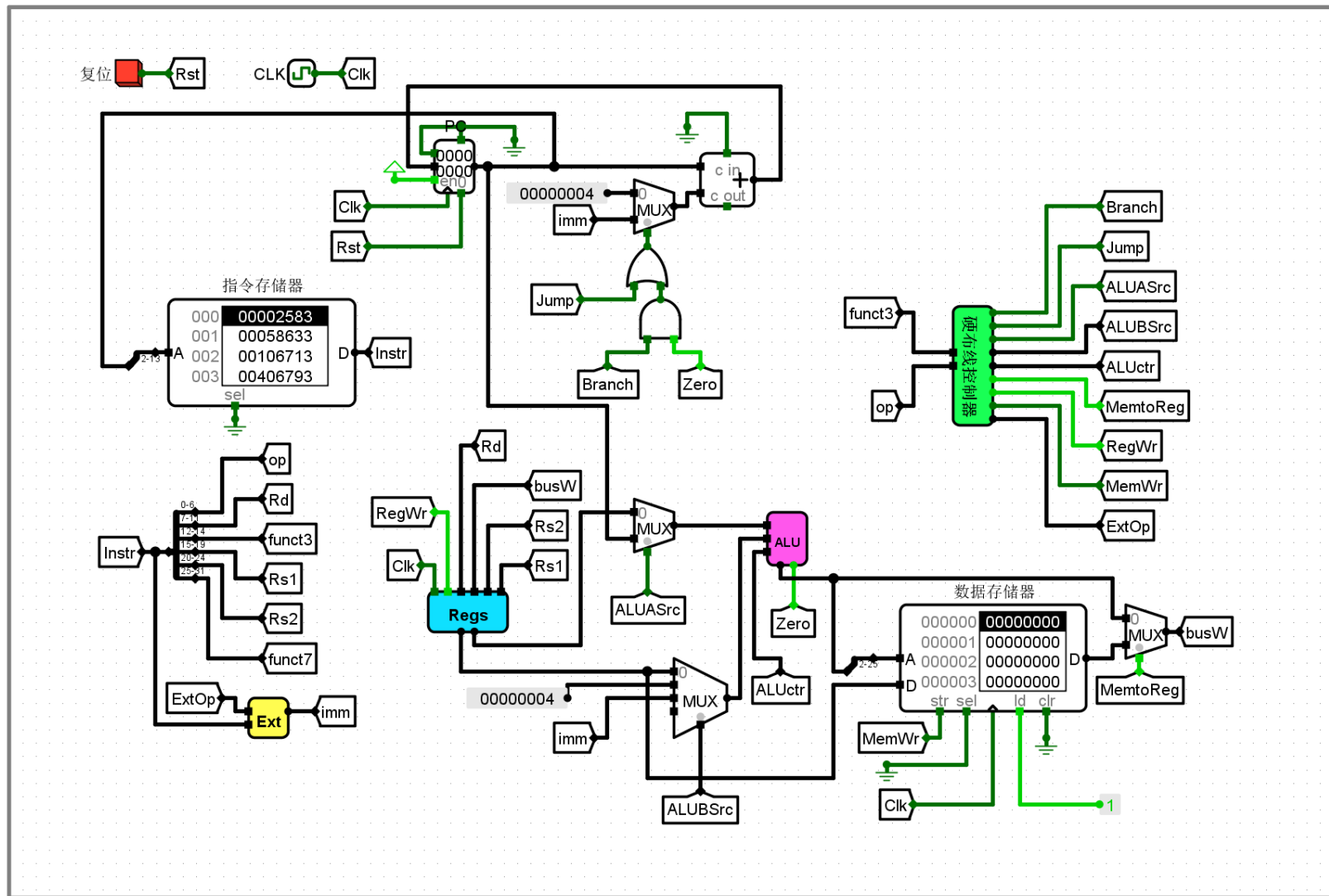
4、单周期 RISC-V 处理器（硬布线控制器） (9条指令) （验证实验）

第四次实验的内容

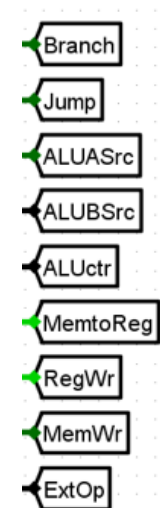
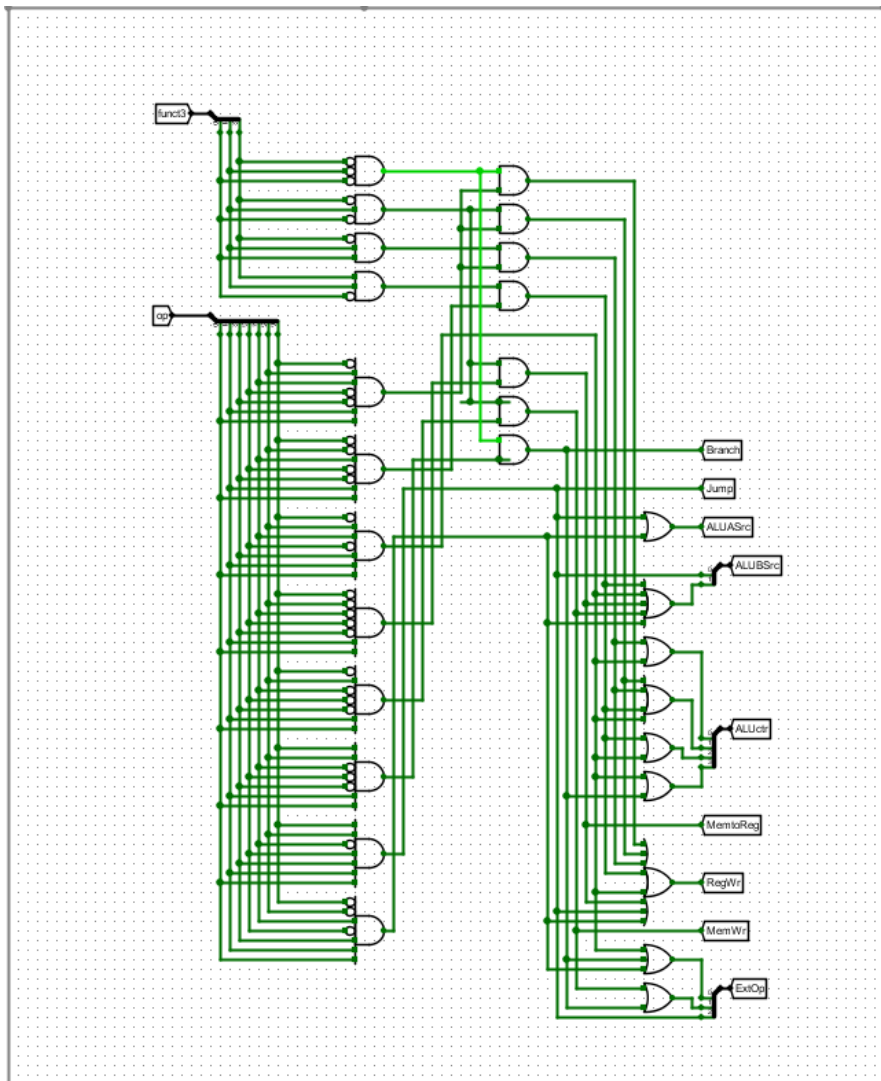
- (1) 单周期 RISC-V 处理器（硬布线控制器）支持的9条指令：

①	<code>add rd,rs1,rs2</code>	； <code>rs1+rs2->rd</code>
②	<code>slt rd,rs1,rs2</code>	； 带符号数比较指令 if <code>rs1<rs2</code> 1-> <code>rd</code> else 0-> <code>rd</code>
③	<code>sltu rd,rs1,rs2</code>	； 无符号数比较指令 if <code>rs1<rs2</code> 1-> <code>rd</code> else 0-> <code>rd</code>
④	<code>ori rd,rs1,imm12</code>	； <code>rs1</code> 或 <code>imm12 -> rd</code>
⑤	<code>lw rd,rs1,imm12</code>	； <code>M[rs1+imm12] -> rd</code>
⑥	<code>lui rd,imm20</code>	； <code>imm20 -> rd</code>
⑦	<code>sw rs2,rs1,imm12</code>	； <code>rs2 -> M[rs1+imm12]</code>
⑧	<code>beq rs1,rs2,imm12</code>	； if <code>rs1=rs2</code> then goto <code>imm12</code>
⑨	<code>jal rd,imm20</code>	； goto <code>imm20</code> <code>PC+4 ->rd</code>

- (2) 单周期 RISC-V 处理器（硬布线控制器）的**数据通路**



硬布线控制器



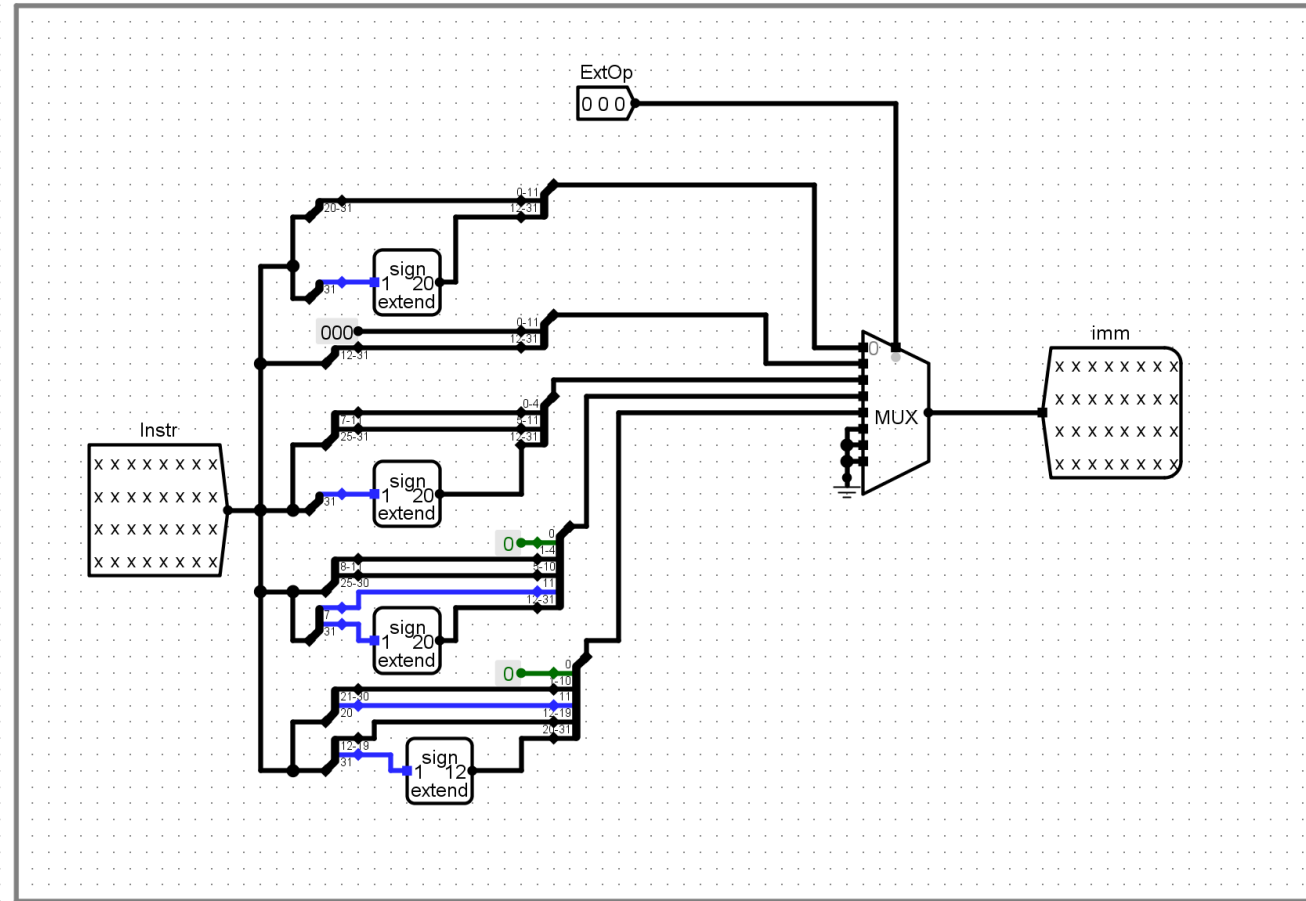
控制信号（9个）

输入：OP（7位）、func3（3位）

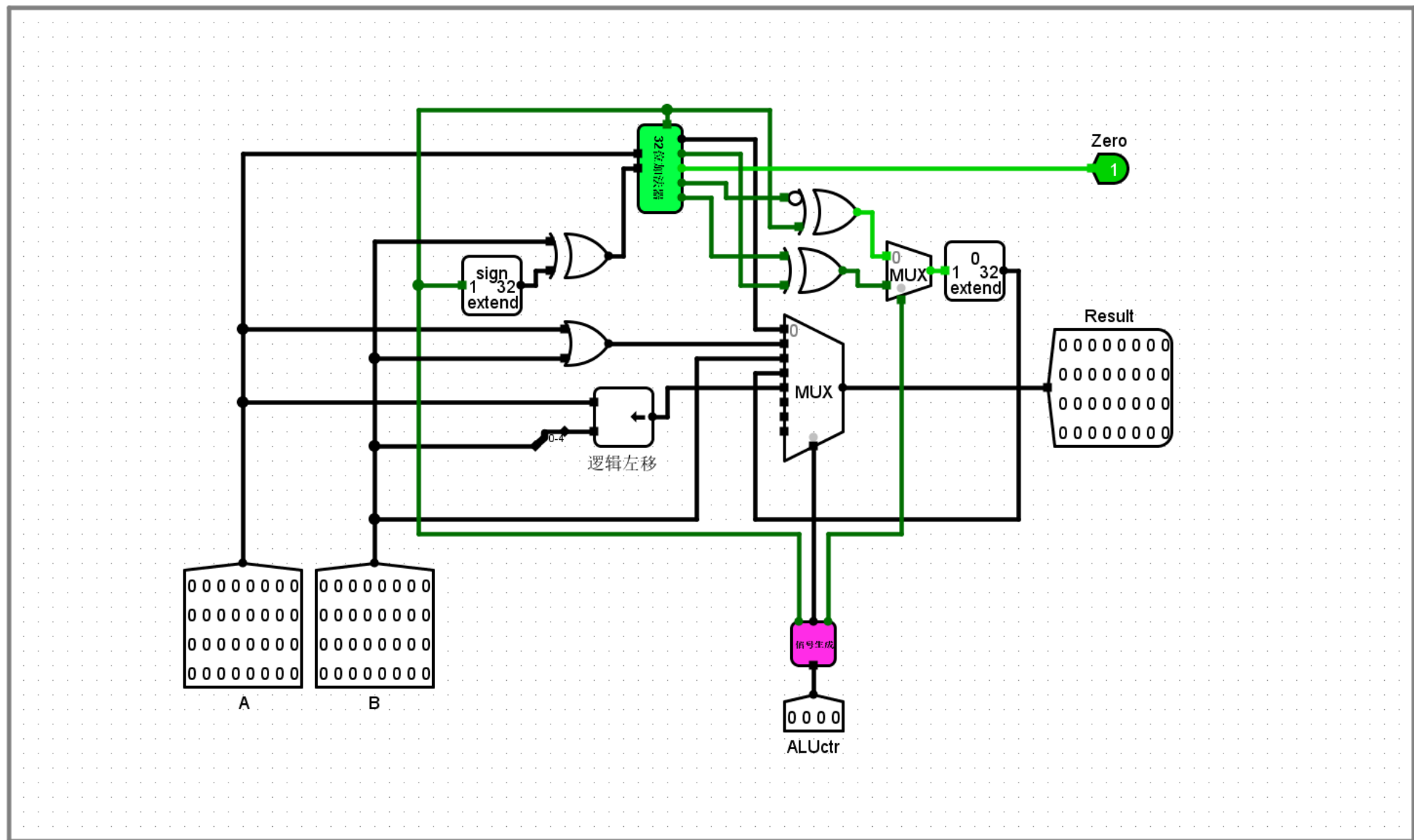
输出：控制信号（9个）

根据指令的OP字段（7位）、func3字段（3位），知道是哪条指令，然后给出相应的控制信号（9个）

立即数扩展部件Ext

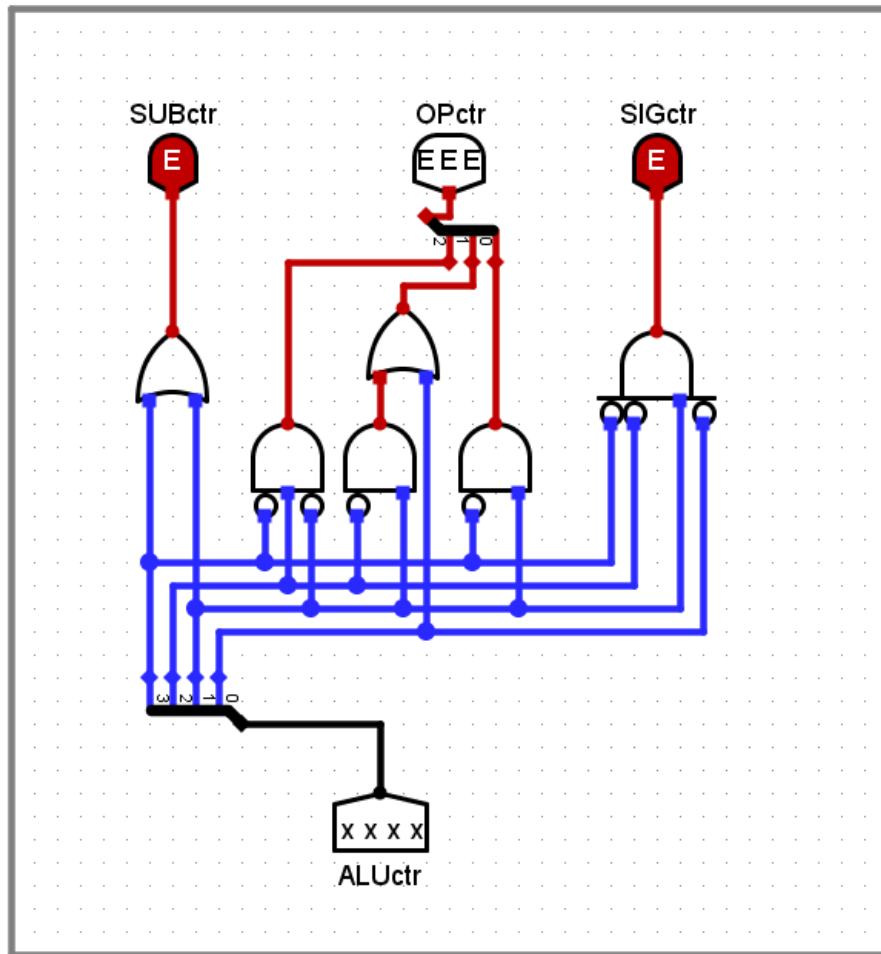


算术逻辑单元ALU



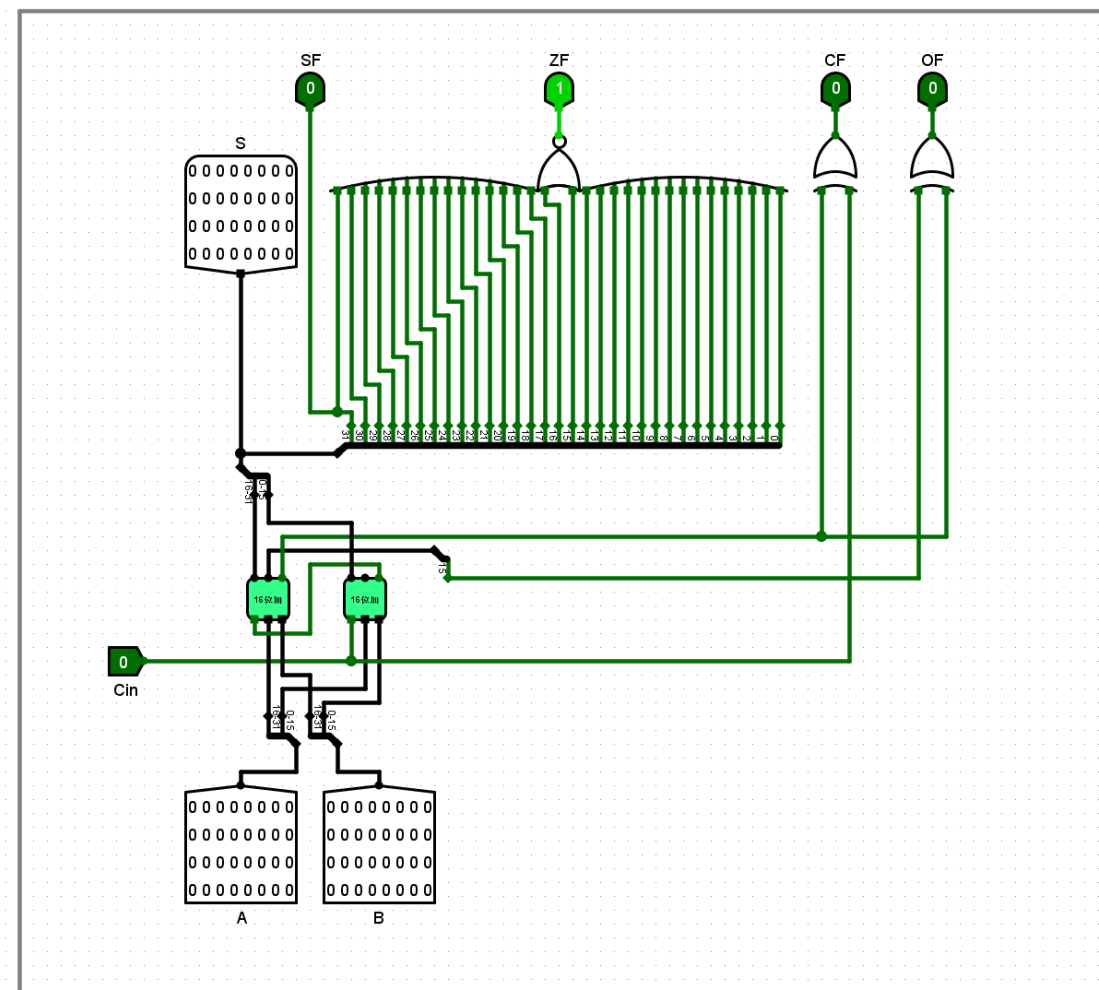
ALU控制信号生成

ALU控制信号生成部件



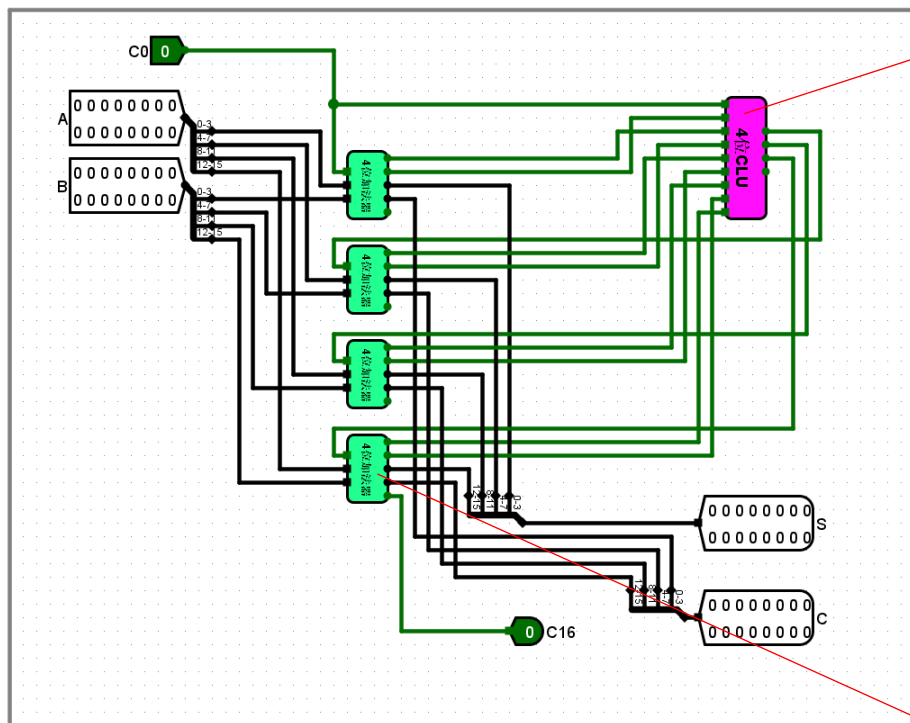
32位加法器

32位加法器

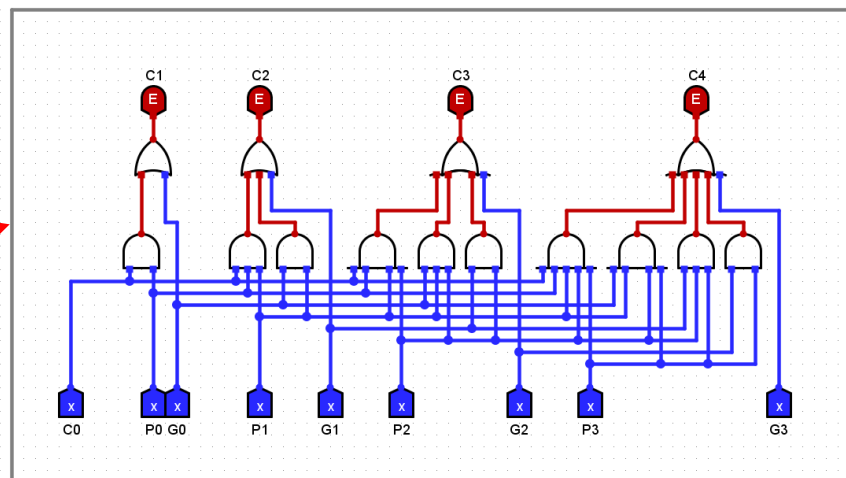


16位加法器

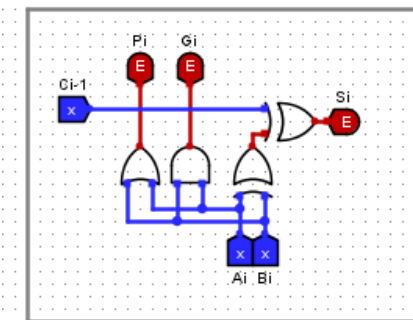
16位加法器



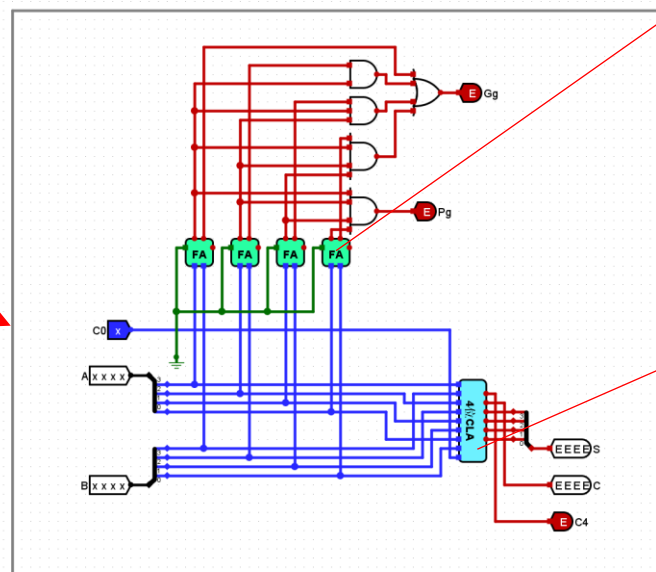
4位先行进位电路CLU



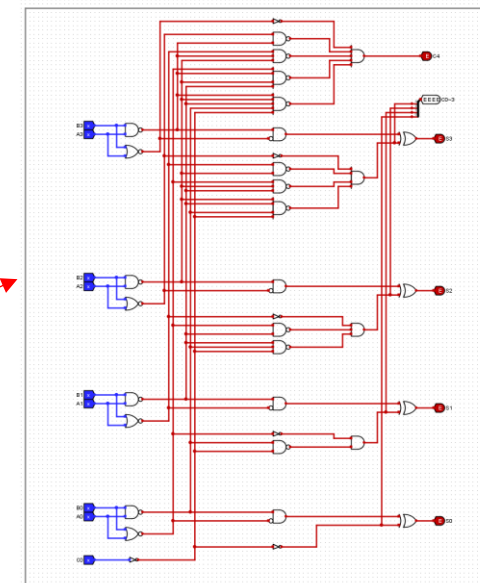
1位全加器FA



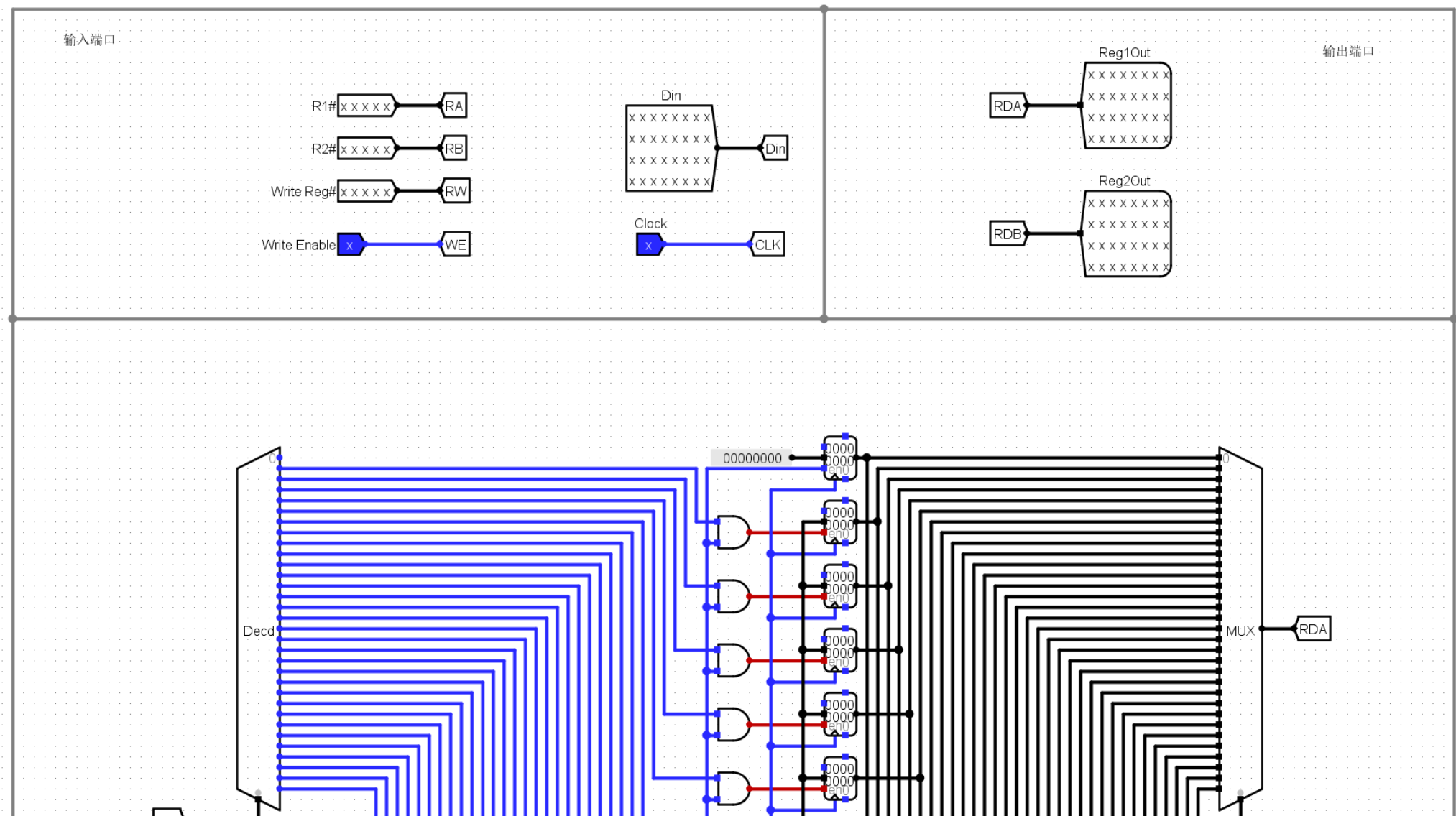
4位加法器



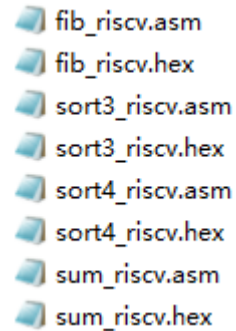
4位快速加法器 CLA



寄存器堆Regs



- (3) **测试程序**：包括求累加和程序、计算费波那契数列程序、排序程序（降序、升序）
 - 先在指令存储器中装入.hex文件，按Ctrl+K，再按“复位”按钮，指令存储器中的内容停止后，按Ctrl+K，观看数据存储器中的内容是否正确？



fib_riscv.asm
fib_riscv.hex
sort3_riscv.asm
sort3_riscv.hex
sort4_riscv.asm
sort4_riscv.hex
sum_riscv.asm
sum_riscv.hex

- (4) 请问：该单周期 RISC-V 处理器的控制器能不能采用**微程序控制器**的方法设计？为什么？
- (5) 请分析单周期 RISC-V处理器（硬布线控制器）的**电路原理**：
 - 包括：数据通路、硬布线控制器、立即数扩展部件Ext、算术逻辑单元ALU等电路。
- (6) 鼓励同学们编写**测试程序**，对**9条指令**的功能进行测试。

二、设计实验

单总线结构 MIPS 处理器（微程序控制器）（增加1条add指令）（6条指令）

单总线结构 MIPS 处理器（微程序控制器） (增加1条add指令) (6条指令) (设计实验)

- 前面的单总线结构 MIPS 处理器（微程序控制器）只支持5条指令，不能运行求累加和程序、计算费波那契数列程序。
- 请对该单总线结构 MIPS 处理器的微程序控制器进行修改，使其支持6条指令（增加1条add指令），并对新增的add指令进行测试。

5条指令

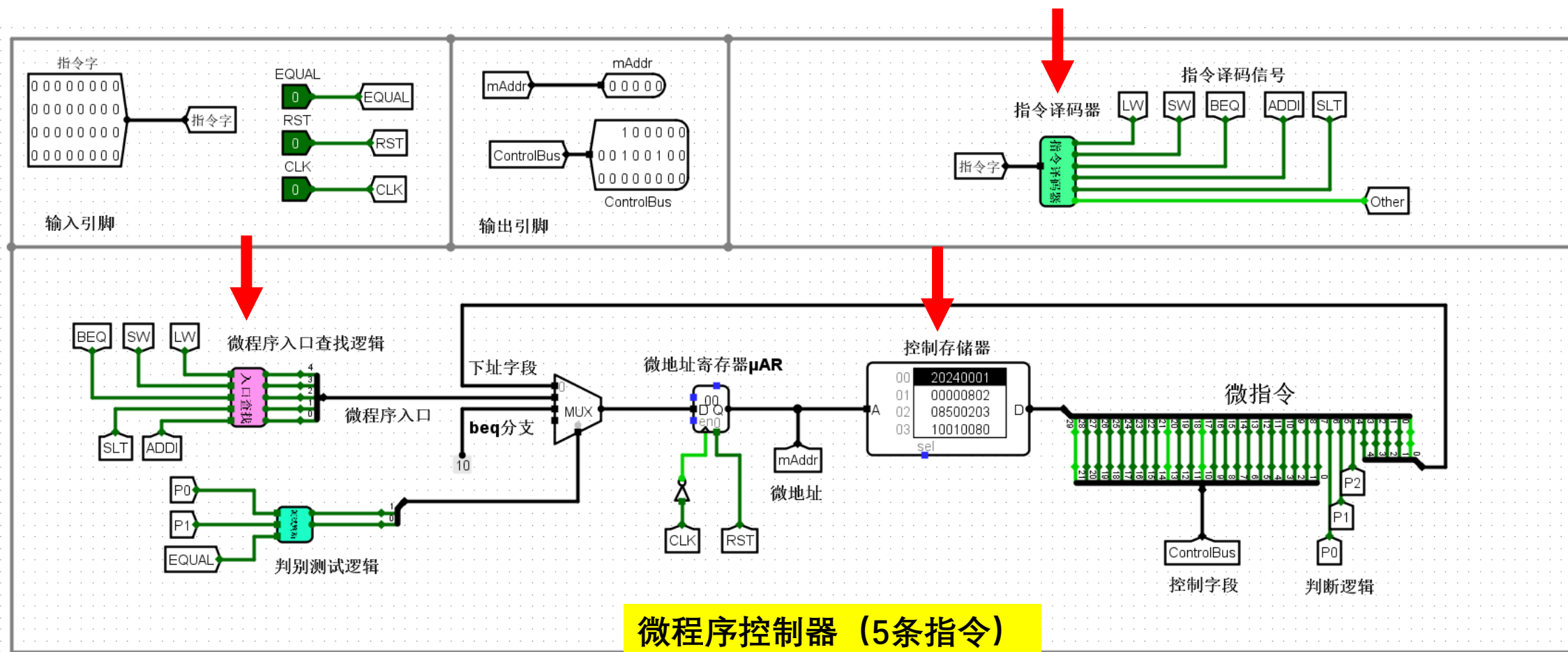
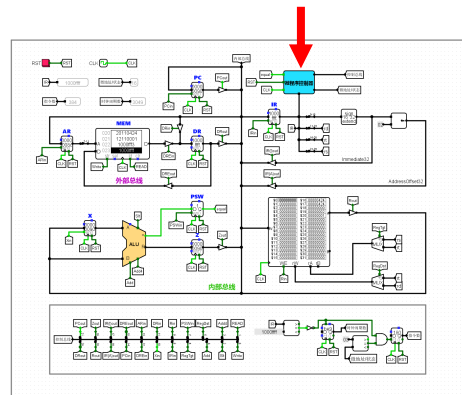
序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 $\leftarrow$ 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 $\leftarrow$ 寄存器
3	beq	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) PC $\leftarrow$ PC + 4 + $\text{SignExt}(imm) \ll 2$	条件分支指令：如果rs = rt，则跳转
4	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
5	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 $\leftarrow$ 寄存器 + 立即数

6条指令

序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 $\leftarrow$ 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 $\leftarrow$ 寄存器
3	beq	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) PC $\leftarrow$ PC + 4 + $\text{SignExt}(imm) \ll 2$	条件分支指令：如果rs = rt，则跳转
4	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
5	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 $\leftarrow$ 寄存器 + 立即数
6	add	add rd,rs,rt	R型	$R[rd] \leftarrow R[rs] + R[rt]$	加法指令：寄存器 $\leftarrow$ 寄存器 + 寄存器

• 设计思路:

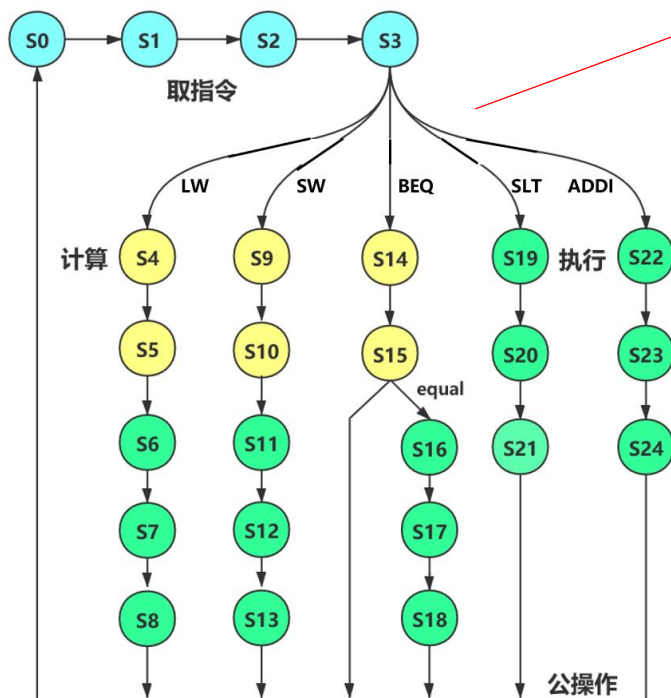
- (1) 数据通路不需要修改, 只需要修改**微程序控制器**。
- (2) 修改**指令译码器**, 增加add指令。
- (3) 修改**微程序入口查找逻辑**, 增加add指令。
- (4) 判别测试逻辑不需要修改。
- (5) 修改控制存储器中的**微程序**, 增加add指令对应的3条微指令。



- 如何修改“微程序入口查找逻辑”？

- (1) 5条指令的微程序入口查找逻辑公式。

5条指令的微程序入口地址



LW=1
SW=1
BEQ=1
SLT=1
ADDI=1

微程序入口地址=4
微程序入口地址=9
微程序入口地址=14
微程序入口地址=19
微程序入口地址=22

微程序入口查找逻辑自动生成 (5条指令) .xlsx

机器指令译码信号					微程序入口地址					
LW	SW	BEQ	SLT	ADDI	入口地址 10进制	S4	S3	S2	S1	S0
1					4	0	0	1	0	0
	1				9	0	1	0	0	1
		1			14	0	1	1	1	0
			1		19	1	0	0	1	1
				1	22	1	0	1	1	0

填写

[illegible]

自动生成

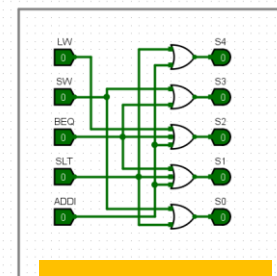
微程序入口查找逻辑公式 (5条指令) .txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
微程序入口查找逻辑 (5条指令) :

S4=SLT+ADDI
S3=SW+BEQ
S2=LW+BEQ+ADDI
S1=BEQ+SLT+ADDI
S0=SW+SLT

拷贝出来

微程序入口查找逻辑自动生成 (5条指令).txt

微程序入口查找逻辑



生成电路

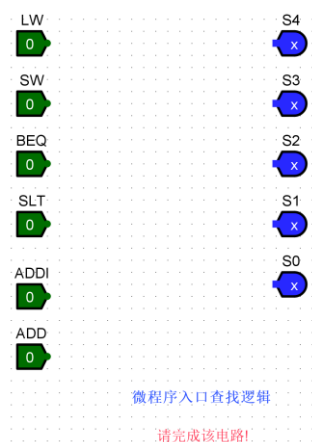
- (2) 请填写“微程序入口查找逻辑自动生成（6条指令）（空表）.xlsx”，并生成6条指令的微程序入口查找逻辑公式。

微程序入口查找逻辑自动生成 (6条指令) (空表) .xlsx

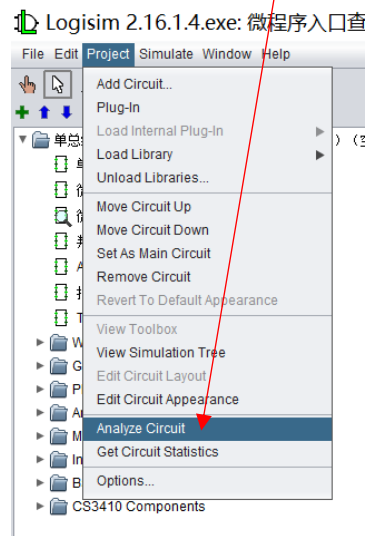
[illegible][illegible]

- (3) 在Logisim上生成逻辑电路。

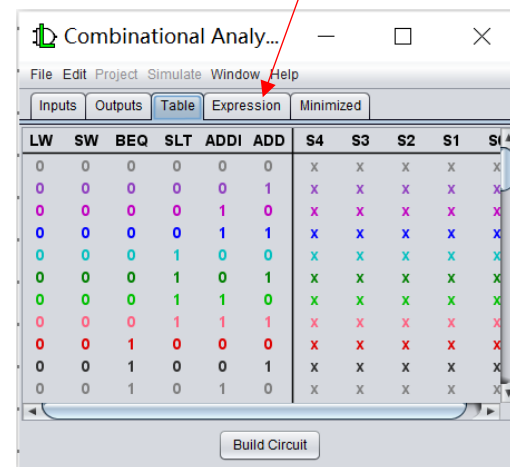
第一步：新建一个电路
(6个输入、5个输出)



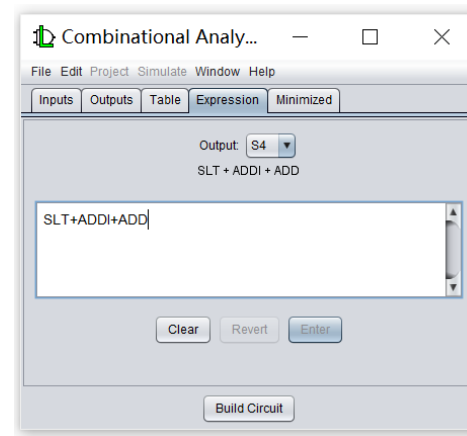
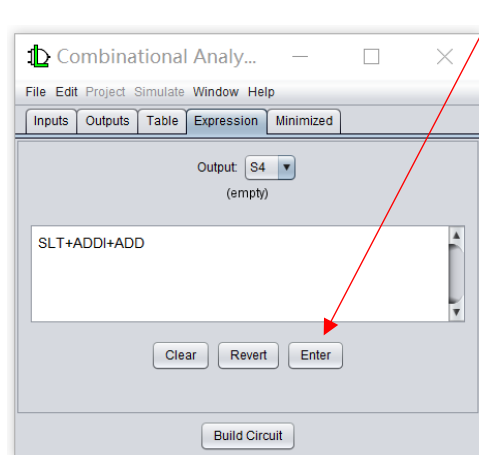
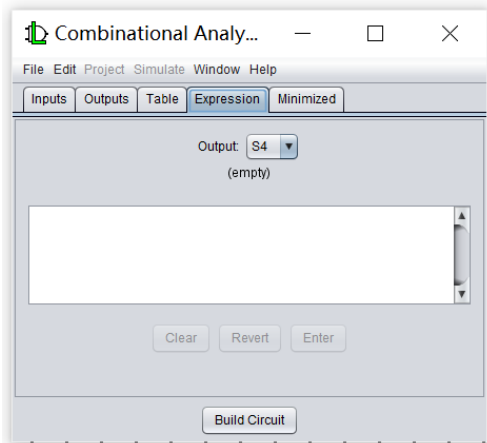
第二步：点击这个菜单



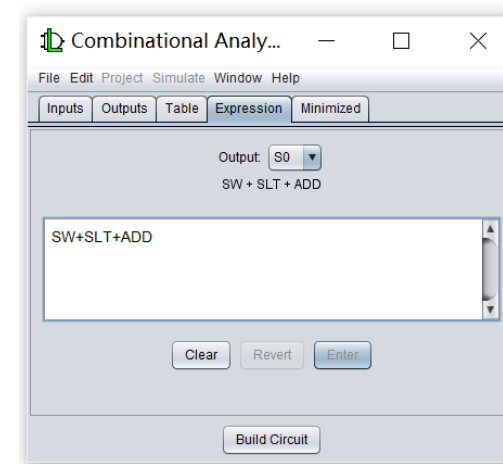
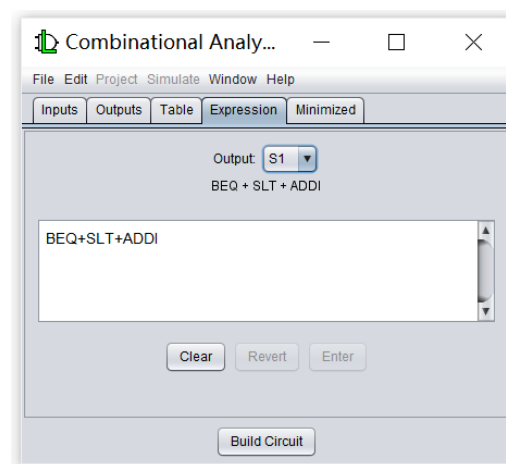
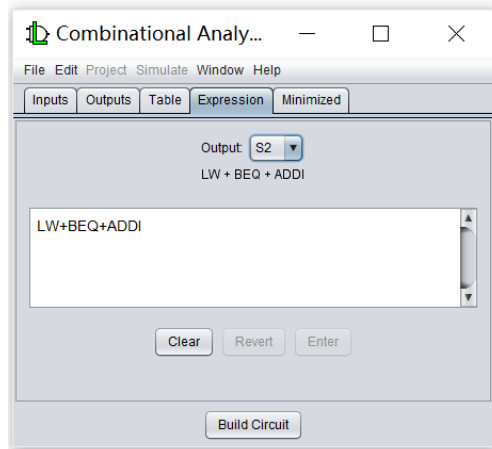
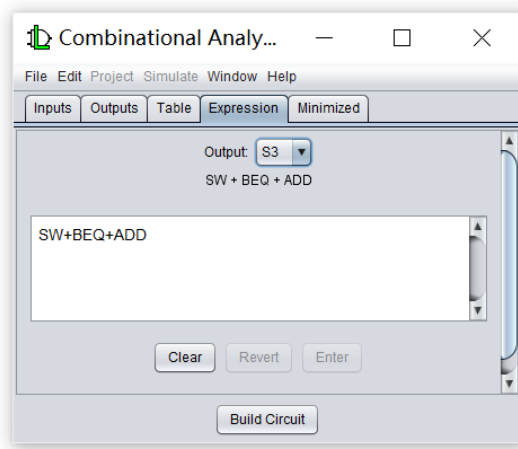
第三步：点击“Expression”



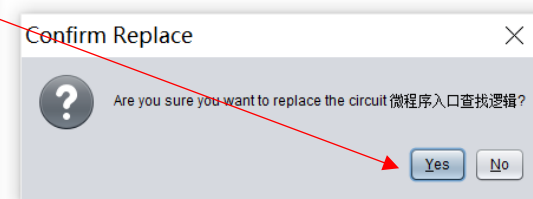
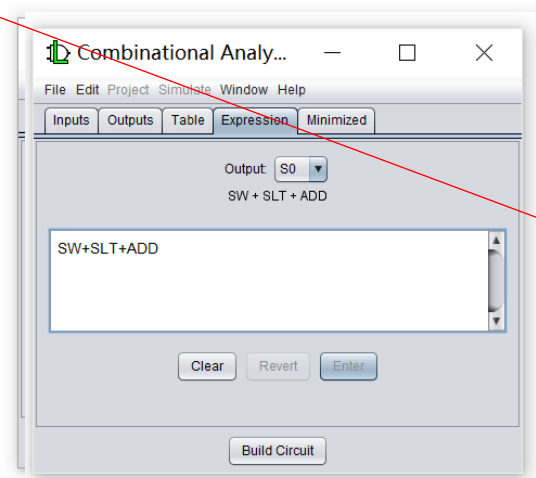
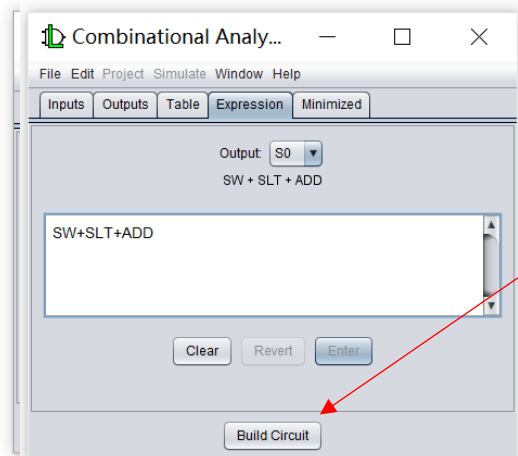
第四步：输入S4的逻辑公式，点击“Enter”



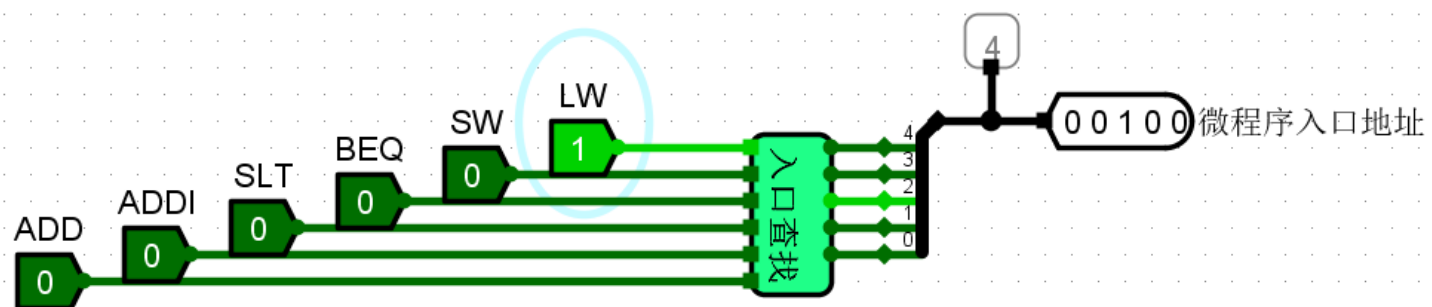
第五步：输入S3、S2、S1、S0的逻辑公式



第六步：点击“Build Circuit”，OK，Yes；即可生成电路



- (4) 测试生成的“微程序入口查找逻辑电路”的正确性。



微程序入口查找逻辑测试电路

- (1) 5条指令的微程序。

微程序 (5条指令) .hex

微程序 (单总线)
文件(F) 编辑(E) 格
v2.0 raw
20240001
802
8500203
10010080
4040005
2001006
8200007
100208
10020000
404000A
200100B
820000C
408400D
800100
404000F
400C040
20040011
1001012
8400000
4040014
4004415
8022000
4040017
2001018
8020000

拷贝出来

- (2) 请填写“微程序自动生成 (add指令) (空表) .xlsx”，生成add指令的微程序，然后添加到5条指令的微程序最后面。

微程序自动生成 (add指令) (空表) .xlsx

[illegible]

• 程序测试：

- (1) 请使用测试程序test2.hex，对设计好的单总线 MIPS 处理器（微程序控制器，支持6条指令）进行测试。

test2.asm - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#单总线结构MIPS处理器测试程序

test2.asm

#测试6条指令 (lw、sw、beq、addi、slt、add)

#程序执行完 RAM存储器的100、101、102单元的内容分别为：8、9、a

100 00000008 00000009 0000000a

main:

```
addi $s0,$zero,8      #s0=8
sw $s0,1024($zero)    #s0=8 保存到(1024, 即100单元)

addi $s0,$zero,9      #s0=9
sw $s0,1028($zero)    #s0=9 保存到(1028, 即101单元)

lw $s1,1024($zero)    #s1=8
lw $s2,1028($zero)    #s2=9

slt $s3,$s1,$s2       # s1 < s2    s3=1

beq $s1,$s2,loop1     #s1不等于s2  执行下一条指令

→ add $s4,$s3,$s2      # s4=1+9=10=a

sw $s4,1032($zero)    #s4=10=a 保存到(1032, 即102单元)

beq $zero,$zero,loop2 #转loop2
```

loop1:

```
addi $s4,$zero,2      #s4=2
sw $s4,1032($zero)    #s4=2 保存到(1032, 即102单元)
```

这条指令不会执行到
这条指令不会执行到

loop2:

```
beq $zero,$zero,loop2 #转loop2    死循环
```

- (2) 在单总线 MIPS 处理器（微程序控制器，支持6条指令）的数据通路上运行求累加和程序、计算费波那契数列程序、排序程序（降序排序、升序排序）。

 fib_mips_bus.asm

```
100 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037
```

 fib_mips_bus.hex

 sort3_mips_bus.asm

```
100 00000005 00000004 00000003 00000002 00000001
```

 sort3_mips_bus.hex

 sort4_mips_bus.asm

```
100 00000001 00000002 00000003 00000004 00000005
```

 sort4_mips_bus.hex

 sum_mips_bus.asm

```
100 00000037
```

 sum_mips_bus.hex

三、挑战性实验

单总线结构 MIPS 处理器（**硬布线控制器**）（增加1条**add指令**）（6条指令）

单总线结构 MIPS 处理器（硬布线控制器）

（增加1条add指令） （6条指令） （挑战性实验）

- 请修改前面的单总线结构 MIPS 处理器（硬布线控制器）（5条指令），使其支持6条指令（增加1条add指令），并对新增的add指令进行测试。

5条指令

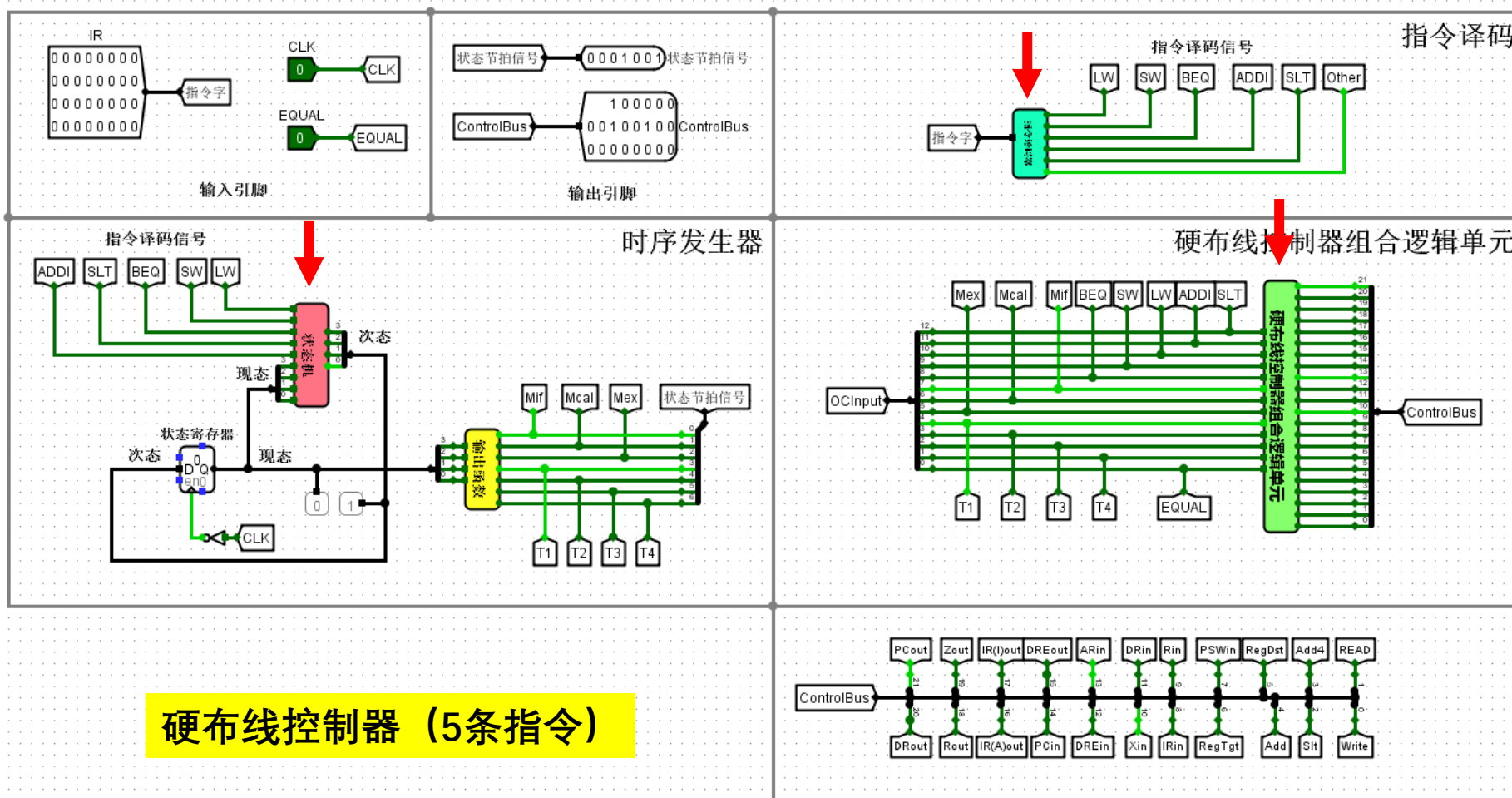
序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 <- 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 <- 寄存器
3	beq	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) PC <- PC + 4 + $\text{SignExt}(imm) \ll 2$	条件分支指令：如果rs = rt，则跳转
4	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
5	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 <- 寄存器 + 立即数

6条指令

序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 <- 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 <- 寄存器
3	beq	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) PC <- PC + 4 + $\text{SignExt}(imm) \ll 2$	条件分支指令：如果rs = rt，则跳转
4	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
5	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 <- 寄存器 + 立即数
6	add	add rd,rs,rt	R型	$R[rd] \leftarrow R[rs] + R[rt]$	加法指令：寄存器 <- 寄存器 + 寄存器

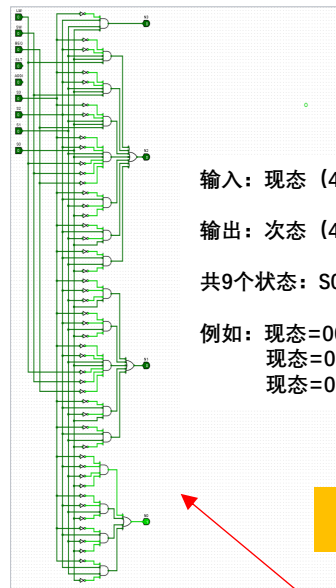
思路:

- (1) 数据通路不需要修改
- (2) 指令译码器需要修改 (与6条指令的微程序控制器的指令译码器相同)
- (3) 状态机需要修改
- (4) 输出函数不需要修改
- (5) 硬布线控制器组合逻辑单元需要修改



如何修改“状态机”的逻辑电路？

- (1) 5条指令的“状态机”逻辑电路。



输入：现态（4位），5个指令译码信号（LW、SW、BEQ、SLT、ADDI）

输出：次态（4位）

共9个状态：S0~S8

例如：现态=0000=S0，次态=0001=S1
现态=0011=S3，SLT=1，次态=0110=S6
现态=0011=S3，LW=1，次态=0100=S4

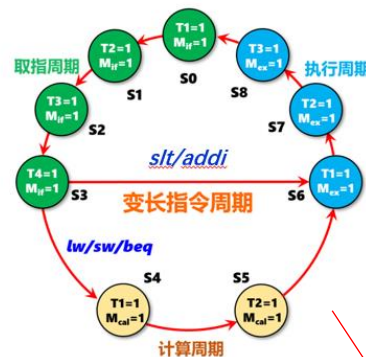
生成电路

状态机逻辑公式 (5条指令) .txt

状态机逻辑公式 (5条指令) .txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
状态机逻辑公式 (5条指令) :

```
N3 = ~S3&S2&S1&S0
N2 = ~S3&~S2&S1&S0&LW+~S3&~S2&S1&S0&SW+~S3&~S2&S1&S0&BEQ+~S3&~S2&S1&S0&SLT+~S3&~S2&S1&S0&ADDI+~S3&S2&~S1&~S0+~S3&S2&~S1&S0+~S3&S2&S1&~S0
N1 = ~S3&~S2&~S1&S0+~S3&~S2&S1&~S0+~S3&~S2&S1&S0&SLT+~S3&~S2&S1&S0&ADDI+~S3&S2&~S1&S0+~S3&S2&S1&~S0
N0 = ~S3&~S2&~S1&~S0+~S3&~S2&S1&~S0+~S3&S2&~S1&~S0+~S3&S2&S1&~S0
```

拷贝出来



1	当前状态(现态)					输入信号					下一状态 (次态)				
2	S3	S2	S1	S0	现态 10进制	LW	SW	BEQ	SLT	ADDI	次态 10进制	N3	N2	N1	N0
3	0	0	0	0	0						1	0	0	0	1
4	0	0	0	1	1				填写		2	0	0	1	0
5	0	0	1	0	2						3	0	0	1	1
6	0	0	1	1	3	1					4	0	1	0	0
7	0	0	1	1	3		1				4	0	1	0	0
8	0	0	1	1	3			1			4	0	1	0	0
9	0	0	1	1	3				1		6	0	1	1	0
10	0	0	1	1	3					1	6	0	1	1	0
11	0	1	0	0	4						5	0	1	0	1
12	0	1	0	1	5						6	0	1	1	0
13	0	1	1	0	6						7	0	1	1	1
14	0	1	1	1	7						8	1	0	0	0
15	1	0	0	0	8						0	0	0	0	0

填写

	S3	S2	S1	S0	LW	SW	BEQ	SLT	ADDI	最小项表达式	N3	N2	N1	N0
1	~S3&	~S2&	~S1&	~S0&						~S3&~S2&~S1&~S0				~S3&~S2&~S1&~S0+
3	~S3&	~S2&	~S1&	S0&						~S3&~S2&~S1&S0				~S3&~S2&~S1&S0+
4	~S3&	~S2&	S1&	~S0&						~S3&~S2&S1&~S0				~S3&~S2&S1&~S0+
5	~S3&	~S2&	S1&	S0&	LW&					~S3&~S2&S1&S0&LW		~S3&~S2&S1&S0&LW+		
6	~S3&	~S2&	S1&	S0&		SW&				~S3&~S2&S1&S0&SW		~S3&~S2&S1&S0&SW+		
7	~S3&	~S2&	S1&	S0&			BEQ&			~S3&~S2&S1&S0&BEQ		~S3&~S2&S1&S0&BEQ+		
8	~S3&	~S2&	S1&	S0&				SLT&		~S3&~S2&S1&S0&SLT		~S3&~S2&S1&S0&SLT+	~S3&~S2&S1&S0&SLT+	
9	~S3&	~S2&	S1&	S0&					ADDI&	~S3&~S2&S1&S0&ADDI		~S3&~S2&S1&S0&ADDI+		
10	~S3&	S2&	~S1&	~S0&						~S3&S2&~S1&~S0		~S3&S2&~S1&~S0+		~S3&S2&~S1&~S0+
11	~S3&	S2&	~S1&	S0&						~S3&S2&~S1&S0		~S3&S2&~S1&S0+	~S3&S2&~S1&S0+	
12	~S3&	S2&	S1&	~S0&						~S3&S2&S1&~S0		~S3&S2&S1&~S0+		~S3&S2&S1&~S0+
13	~S3&	S2&	S1&	S0&						~S3&S2&S1&S0	~S3&S2&S1&S0+			
14	S3&	~S2&	~S1&	~S0&						S3&~S2&~S1&~S0				
15														
16														
17														
18														
19														
20														
21														
22														
23														
24														
25														
26														
27														
28														
29														
30														
31											~S3&S2&S1&S0			

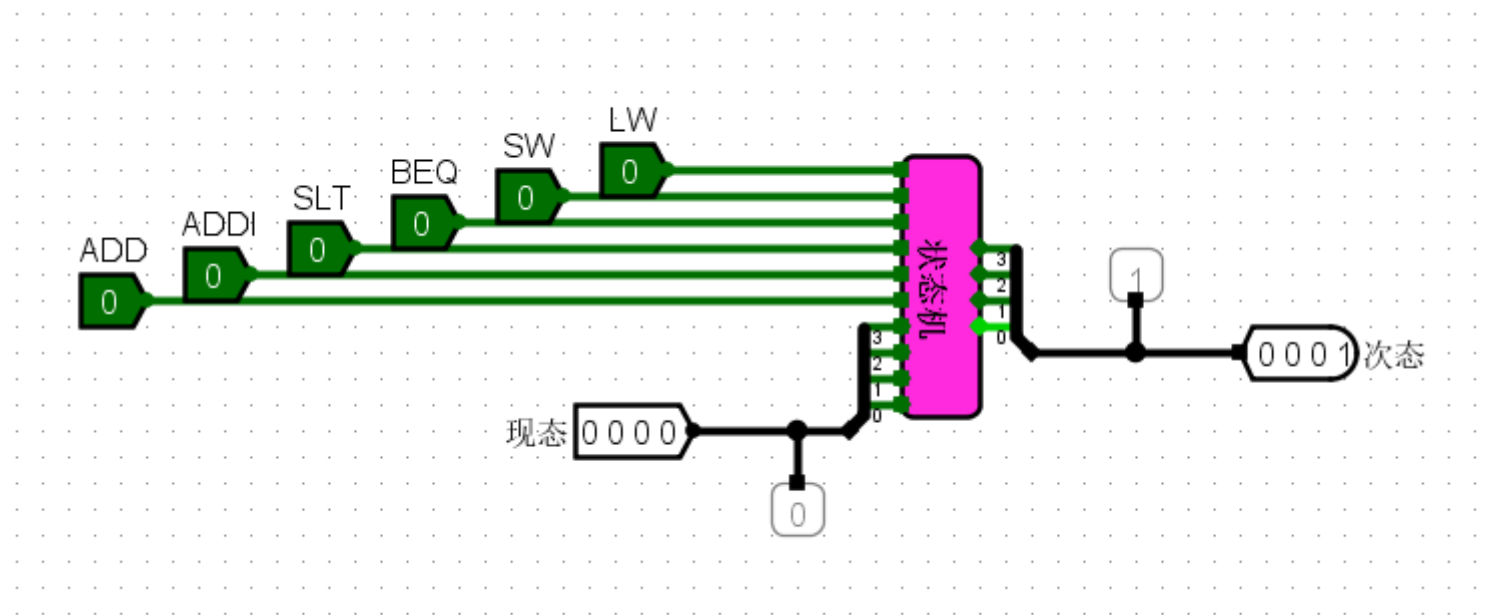
自动生成

- (2) 请填写“状态机自动生成 (6条指令) (空表) .xlsx”，并生成状态机逻辑公式，然后在Logisim上生成逻辑电路。

状态机自动生成 (6条指令) (空表) .xlsx

[illegible][illegible]

- (3) 测试生成的“状态机”电路的正确性。



- 如何修改“硬布线控制器组合逻辑单元”的逻辑电路？

- (1) 5条指令的“硬布线控制器组合逻辑单元”逻辑电路。

硬布线控制器组合逻辑自动生成 (5条指令) .xlsx

[illegible]

填写

拷贝出来

生成电路

硬布线控制器组合逻辑单元

输入: Mif、Mcal、Mex, T1、T2、T3、T4, LW、SW、BEQ、SLT、ADDI, EQUAL

输出：22个控制信号

这里的Mif、Mcal、Mex，相当于图6.43中的M1~Mi

这里的T1、T2、T3、T4，相当于图6.43中的T1~Tk

这里的LW、SW、BEQ、SLT、ADDI，相当于图6.43中的译码信号I1~Im

这里的EQUAL，相当于图6.43中的反馈信号B1~Bj

自动生成

硬布线控制器组合逻辑公式 (5条指令).txt

- 

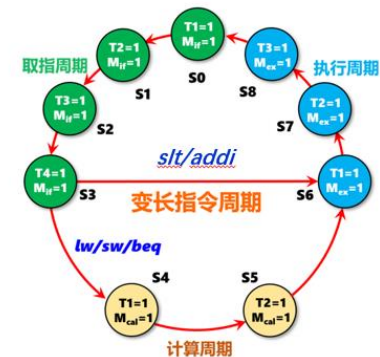
硬布线控制器组合逻辑自动生成（6条指令）（空表）.xlsx

[illegible]

- 输出函数电路的自动生成 (6条指令的输出函数电路与5条指令相同, **不需要修改**)

填写

当前状态(现态)					输出						
S3	S2	S1	S0	现态 10进制	Mif	Mcal	Mex	T1	T2	T3	T4
0	0	0	0	0	1			1			
0	0	0	1	1	1				1		
0	0	1	0	2	1					1	
0	0	1	1	3	1						1
0	1	0	0	4		1		1			
0	1	0	1	5		1			1		
0	1	1	0	6			1	1			
0	1	1	1	7			1		1		
1	0	0	0	8			1			1	

[illegible]

输出函数逻辑公式.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
输出函数逻辑公式 (5条指令):

Mif= $\sim S3\&\sim S2\&\sim S1\&\sim S0+\sim S3\&\sim S2\&\sim S1\&S0+\sim S3\&\sim S2\&S1\&\sim S0+\sim S3\&\sim S2\&S1\&S0$

Mcal= $\sim S3\&S2\&\sim S1\&\sim S0+\sim S3\&S2\&\sim S1\&S0$

Mex= $\sim S3\&S2\&S1\&\sim S0+\sim S3\&S2\&S1\&S0+S3\&\sim S2\&\sim S1\&\sim S0$

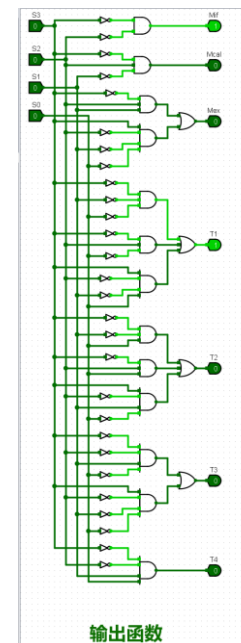
T1= $\sim S3\&\sim S2\&\sim S1\&\sim S0+\sim S3\&S2\&\sim S1\&\sim S0+\sim S3\&S2\&S1\&\sim S0$

T2= $\sim S3\&\sim S2\&\sim S1\&S0+\sim S3\&S2\&\sim S1\&S0+\sim S3\&S2\&S1\&S0$

T3= $\sim S3\&\sim S2\&S1\&\sim S0+S3\&\sim S2\&\sim S1\&\sim S0$

T4= $\sim S3\&\sim S2\&S1\&S0$

拷贝出来



自动生成

生成电路

• 程序测试:

- (1) 请使用测试程序test2.hex, 对设计好的单总线 MIPS 处理器（硬布线控制器，支持6条指令）进行测试。

test2.asm - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

#单总线结构MIPS处理器测试程序

test2.asm

#测试6条指令 (lw、sw、beq、addi、slt、add)

#程序执行完 100、101、102单元的内容分别为: 8、9、a

100 00000008 00000009 0000000a

main:

addi \$s0,\$zero,8 #s0=8
sw \$s0,1024(\$zero) #s0=8 保存到(1024, 即100单元)

addi \$s0,\$zero,9 #s0=9
sw \$s0,1028(\$zero) #s0=9 保存到(1028, 即101单元)

lw \$s1,1024(\$zero) #\$s1=8
lw \$s2,1028(\$zero) #\$s2=9

slt \$s3,\$s1,\$s2 # s1 < s2 s3=1

beq \$s1,\$s2,loop1 #s1不等于s2 执行下一条指令

➔ add \$s4,\$s3,\$s2 # s4=1+9=10=a

sw \$s4,1032(\$zero) #s4=10=a 保存到(1032, 即102单元)

beq \$zero,\$zero,loop2 #转loop2

loop1:

addi \$s4,\$zero,2 #s4=2 这条指令不会执行到
sw \$s4,1032(\$zero) #s4=2 保存到(1032, 即102单元) 这条指令不会执行到

loop2:

beq \$zero,\$zero,loop2 #转loop2 死循环

- (2) 在单总线 MIPS 处理器（硬布线控制器，支持6条指令）的数据通路上运行求累加和程序、计算费波那契数列程序、排序程序（降序排序、升序排序）。

 fib_mips_bus.asm

100 00000000 00000001 00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037

 fib_mips_bus.hex

 sort3_mips_bus.asm

100 00000005 00000004 00000003 00000002 00000001

 sort3_mips_bus.hex

 sort4_mips_bus.asm

100 00000001 00000002 00000003 00000004 00000005

 sort4_mips_bus.hex

 sum_mips_bus.asm

100 00000037

 sum_mips_bus.hex

实验要求

- 完成验证实验，并将实验结果通过截屏的方式，复制到实验报告上；并对有关电路原理进行分析
- 完成设计实验，设计实验电路命名为：
 - 单总线 MIPS 处理器-微程序.circ
- 鼓励同学们完成挑战性实验（完成的加8分），挑战性实验电路命名为：
 - 单总线MIPS 处理器-硬布线.circ

实验报告文件提交要求

- 1、请按照实验报告模板（**厦门大学计算机组成原理实验报告样本（第5次实验）.docx**），撰写实验报告（Word版本），实验报告命名为：**学号+姓名+第5次实验报告**
- 2、请将实验报告和设计文件打包为1个压缩文件，压缩文件命名为：**学号+姓名+第5次实验.zip**
- 3、将压缩文件上传到**数字化教学平台**，第5次实验报告提交**截止时间参平台**